

SportPlan — Dossier technique et théorique du projet

Application de gestion et de suivi de séances sportives — Projet de fin de session

Sommaire

1. Introduction
2. Contexte théorique
3. Stack technique détaillée
4. Architecture générale du projet
5. Modèle de données
6. Sécurité : mise en œuvre concrète
7. Revue de code fichier par fichier (backend puis frontend)
8. Tests automatisés
9. Rôles et permissions — synthèse
10. Choix de conception et limites connues
11. Pistes d'évolution
12. Conclusion

1.1 Présentation générale

SportPlan est une application web permettant à un sportif de planifier ses séances d'entraînement, de s'y inscrire, de suivre ses performances et son assiduité, de se fixer des objectifs et de tenir un journal d'entraînement. Un coach peut créer des séances, en assurer le suivi de présence et consulter la progression des sportifs qu'il encadre. Un administrateur supervise l'ensemble des comptes.

L'application est construite selon une architecture **client-serveur en trois couches** : une interface web (client riche, *Single Page Application*) écrite en Angular, une API HTTP écrite en Python avec FastAPI, et une base de données relationnelle SQL Server. Le client et le serveur sont deux projets totalement indépendants, qui ne communiquent que par des requêtes HTTP/JSON : le frontend ne connaît jamais la base de données, et le backend ne connaît jamais l'interface graphique. Cette séparation stricte est elle-même un concept théorique central de ce rapport (voir §2.1) et conditionne l'organisation de tout le code.

1.2 Objectif de ce document

Ce document a deux objectifs :

1. **Expliquer la théorie** derrière chaque choix technique du projet — pas seulement *quoi* a été utilisé, mais *pourquoi* ce concept existe, quel problème il résout, et comment il s'articule avec les autres. C'est la partie la plus développée du document (chapitre 2 et suivants).
2. **Documenter le code fichier par fichier**, backend puis frontend, afin de constituer une référence exhaustive de l'implémentation réelle du projet (chapitre 7).

1.3 Public cible et besoin

Une personne qui s'entraîne régulièrement doit souvent jongler entre plusieurs outils : un agenda pour ses séances, un carnet ou un tableur pour ses performances, une application distincte pour ses objectifs. Cette dispersion de l'information nuit au suivi de la progression et, par conséquent, à la motivation. SportPlan centralise ces besoins dans une seule application, avec une vue adaptée à chaque rôle (sportif, coach, admin).

2. Contexte théorique

02

Cette section explique, indépendamment du code, les concepts d'ingénierie logicielle mobilisés par le projet. Chaque sous-section part d'un problème général avant de montrer comment SportPlan l'aborde.

2.1 Architecture client-serveur et séparation des responsabilités



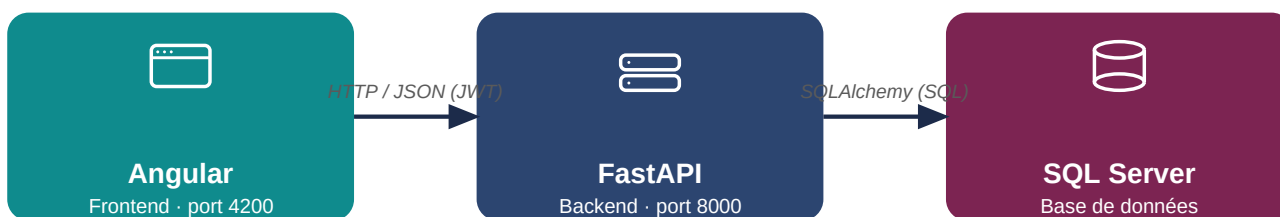
Le problème général. Une application qui mélange interface graphique, logique métier et accès aux données dans un seul programme monolithique devient rapidement difficile à faire évoluer : changer l'apparence risque de casser une règle métier, changer de base de données oblige à réécrire l'affichage, et il est impossible de réutiliser la logique métier pour un autre client (une application mobile, par exemple).

La solution théorique : l'architecture en couches (*layered architecture*). On isole les responsabilités dans des couches qui ne communiquent qu'à travers des interfaces bien définies :

- une couche de **présentation** (l'interface utilisateur),
- une couche **applicative / métier** (les règles de gestion),
- une couche d'**accès aux données** (la persistance).

Chaque couche ne dépend que de la couche immédiatement inférieure et ignore tout de la couche supérieure. C'est le principe de **couplage faible** (*loose coupling*) : le frontend peut être entièrement réécrit (par exemple remplacé par une application mobile) sans toucher au backend, tant que le contrat d'API (les routes HTTP et le format JSON échangé) reste identique.

Application dans SportPlan. Le projet pousse cette séparation au niveau maximal : la couche présentation (Angular) et la couche applicative + données (FastAPI + SQL Server) sont deux **processus séparés**, communiquant uniquement par le réseau via une **API REST** (voir §2.2). C'est la forme la plus stricte de couplage faible : même un changement de langage de programmation d'un côté n'affecterait pas l'autre.



2.2 Le style architectural REST



Le problème général. Pour qu'un client et un serveur développés indépendamment puissent communiquer, il faut un protocole de dialogue standardisé, compréhensible par n'importe quel client (navigateur, application mobile, script) sans connaître les détails internes du serveur.

La théorie REST (*Representational State Transfer*). REST est un style d'architecture (pas un protocole) défini par un ensemble de contraintes :

- **Ressources** : chaque concept métier (une séance, un utilisateur, une participation) est une *ressource* identifiée par une URL (`/sessions/{id}`, `/users/{id}`...).
- **Verbes HTTP normalisés** pour exprimer l'action sur la ressource : `GET` (lire, sans effet de bord), `POST` (créer), `PATCH` (modifier partiellement), `DELETE` (supprimer). SportPlan suit rigoureusement cette convention (voir le détail des routeurs au chapitre 7.1.6).
- **Sans état (*stateless*)** : chaque requête contient toute l'information nécessaire à son traitement (ici, le jeton JWT dans l'en-tête `Authorization`) ; le serveur ne conserve aucune session entre deux requêtes. C'est ce qui permet de faire tourner plusieurs instances du backend derrière un répartiteur de charge sans les synchroniser.
- **Codes de statut HTTP** comme vocabulaire d'erreur standard : `200/201` (succès), `400` (donnée invalide), `401` (non authentifié), `403` (non autorisé), `404` (introuvable), `409` (conflit métier, ex. séance déjà complète). Ce vocabulaire est repris systématiquement dans les routeurs FastAPI du projet.
- **Représentation en JSON** : chaque ressource est échangée sous forme d'un objet JSON, dont la forme est contractualisée côté serveur par un schéma Pydantic (voir §2.4).

2.3 Le patron MVC et son adaptation à une API



Le problème général. Même à l'intérieur d'une seule couche applicative, mélanger la réception de la requête, la validation, la règle métier et l'accès aux données dans une seule fonction géante rend le code illisible et impossible à tester unitairement.

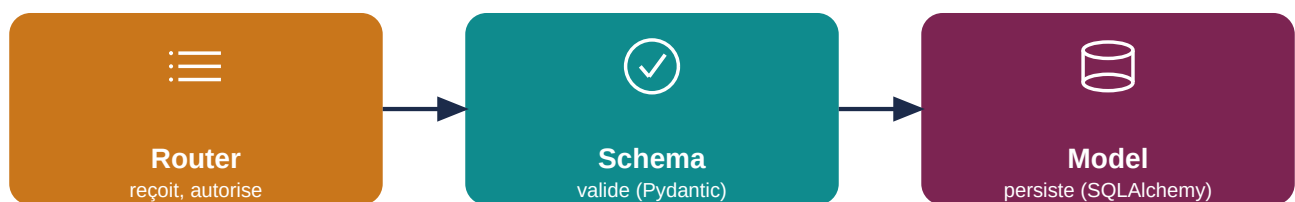


La théorie : Modèle-Vue-Contrôleur (MVC). Le patron MVC répartit un traitement en trois responsabilités : le **modèle** (les données et les règles qui leur sont propres), la **vue** (la présentation), et le **contrôleur** (le chef d'orchestre qui reçoit une requête, appelle le modèle, puis choisit la vue).

Adaptation côté backend (API only, sans vue serveur). Une API REST ne produit pas de vue HTML : la « vue » est déléguée entièrement au frontend. Le backend adapte donc MVC en trois couches très proches de son esprit :

Rôle MVC	Équivalent SportPlan	Dossier
Contrôleur	Reçoit la requête HTTP, orchestre	app/routers/
Modèle (validation d'E/S)	Définit la forme des données échangées	app/schemas/
Modèle (persistance)	Représente une table de la base	app/models/

Le flux d'une requête est donc : *routeur* (reçoit, vérifie les autorisations) -> *schema* (valide la forme de l'entrée) -> manipulation du *model* via la session SQLAlchemy -> retour d'un *schema* de sortie. Cette séparation permet, par exemple, de changer la structure de la table SQL (*models*) sans changer le contrat exposé au client (*schemas*), ou inversement.



Adaptation côté frontend. Angular applique une variante du même principe : le **composant** (`*.component.ts`) joue le rôle de contrôleur + vue (son template HTML), et le **service** (`*.service.ts`, injectable, `providedIn: 'root'`) joue le rôle de modèle / accès aux données, en encapsulant les appels HTTP vers l'API. Un service est **partagé** par tous les composants qui en ont besoin grâce à l'**injection de dépendances** (voir §2.8).

2.4 Mapping objet-relationnel (ORM) et SQLAlchemy

! Le problème général. Une base de données relationnelle stocke des lignes dans des tables, alors qu'un programme orienté objet manipule des instances de classes. Écrire à la main le SQL de chaque requête (`SELECT`, `INSERT`, gestion des jointures, échappement des paramètres contre les injections SQL) pour chaque opération est répétitif et source d'erreurs de sécurité.

La théorie : l'ORM (*Object-Relational Mapping*). Un ORM fait correspondre automatiquement une classe à une table, un attribut à une colonne, et une relation entre classes à une clé étrangère. Le développeur manipule des objets ; l'ORM traduit ces manipulations en SQL. Deux bénéfices majeurs : (1) le code est portable entre moteurs de bases de données différents, (2) les requêtes paramétrées générées par l'ORM éliminent structurellement le risque d'injection SQL par concaténation de chaînes.

SQLAlchemy dans SportPlan. Chaque fichier de `backend/app/models/` définit une classe qui hérite d'une Base déclarative SQLAlchemy ; chaque attribut de classe déclaré avec `Column(...)` devient une colonne SQL, et chaque `relationship(...)` matérialise une association entre deux tables (par exemple une séance a plusieurs participations). La **session** SQLAlchemy (`Session`, injectée via `Depends(get_db)` dans chaque route, cf. §2.8) implémente le patron **Unit of Work** : elle accumule les modifications en mémoire et les

traduit en une transaction SQL unique au moment de `db.commit()`, garantissant que plusieurs écritures liées réussissent ou échouent ensemble (propriété d'**atomicité**, l'un des critères ACID des transactions).

2.5 Validation des données et le patron DTO (Pydantic)



Le problème général. Une API ne doit jamais faire confiance aux données envoyées par un client : un champ manquant, un type incorrect, une valeur hors limites (une date de naissance dans le futur, un score négatif...) doivent être rejetés *avant* d'atteindre la logique métier ou la base de données.

La théorie : le DTO (*Data Transfer Object*) et la validation déclarative. Plutôt que d'écrire des `if` de validation dispersés dans chaque route, on définit un objet dont la seule responsabilité est de décrire la forme attendue des données (types, contraintes, champs obligatoires/optionnels) et qui se valide lui-même à la construction.

Pydantic dans SportPlan. Chaque fichier de `backend/app/schemas/` définit des classes qui héritent de `BaseModel` (Pydantic), avec des annotations de type Python standard (`int`, `str`, `datetime`...) et des contraintes (`Field(gt=0)`, validateurs personnalisés). FastAPI utilise ces schémas à deux moments : en entrée, il déserialise et valide automatiquement le corps JSON de la requête contre le schéma déclaré dans la signature de la route (sinon HTTP 422 est renvoyé automatiquement, sans code applicatif à écrire) ; en sortie, le paramètre `response_model` du décorateur de route garantit que seuls les champs déclarés du schéma de sortie sont sérialisés — même si l'objet SQLAlchemy renvoyé par erreur contenait des champs sensibles (comme un mot de passe haché), ils ne fuiraient jamais vers le client. C'est un mécanisme de sécurité par construction, pas seulement de confort.

2.6 Authentification, autorisation et sécurité

Trois notions à ne pas confondre régissent la sécurité de l'application : l'**authentification** (qui êtes-vous ?), l'**autorisation** (qu'avez-vous le droit de faire ?), et la **protection des secrets** (comment les mots de passe et jetons sont-ils protégés en cas de fuite de la base ?).

2.6.1 Hachage des mots de passe (Argon2)



Le problème. Si la base de données est compromise, les mots de passe des utilisateurs ne doivent jamais être lisibles, ni même reconstituables raisonnablement.



La théorie. On ne *chiffre* pas un mot de passe (le chiffrement est réversible avec la bonne clé) : on le **hache**. Une fonction de hachage cryptographique transforme n'importe quelle entrée en une empreinte de taille fixe, de façon déterministe mais **non réversible** : on ne peut pas retrouver le mot de passe à partir de l'empreinte, on ne peut que vérifier qu'un mot de passe candidat produit la même empreinte. Un simple hachage rapide (SHA-256 par exemple) reste vulnérable à des attaques par force brute massivement parallélisées sur GPU. **Argon2** (vainqueur de la *Password Hashing Competition* de 2015) est une fonction de hachage conçue spécifiquement pour être **lente et coûteuse en mémoire**, rendant le calcul massivement parallèle économiquement inefficace pour un attaquant. Chaque hachage intègre aussi un **sel** aléatoire unique, empêchant les attaques par table précalculée (*rainbow tables*) et garantissant que deux utilisateurs avec le même mot de passe obtiennent des empreintes différentes.

2.6.2 Authentification par jeton (JWT)



Le problème. Une API REST étant sans état (§2.2), le serveur ne peut pas mémoriser « cet utilisateur est connecté » entre deux requêtes comme le ferait une session serveur classique. Il faut un mécanisme qui prouve l'identité de l'appelant à *chaque* requête, sans consultation d'une base de sessions côté serveur.



La théorie : JSON Web Token. Un JWT est une chaîne composée de trois parties séparées par des points et encodées en Base64 : `en-tête.charge_utile.signature`. La charge utile (*payload*) contient des informations (l'identifiant de l'utilisateur, une date d'expiration, un type de jeton) ; la signature est calculée par le serveur avec une clé secrète (`SECRET_KEY`) et garantit que le contenu n'a pas été modifié — sans cette clé, il est impossible de forger un jeton valide ou d'en modifier le contenu sans invalider la signature. Le serveur n'a donc rien à stocker : il valide un jeton reçu en recalculant la signature.

Access token et refresh token. SportPlan utilise deux jetons aux durées de vie différentes : un **jeton d'accès** (courte durée, ex. 30 minutes, envoyé à chaque requête protégée) et un **jeton de rafraîchissement** (durée plus longue, ex. 7 jours, utilisé uniquement pour obtenir un nouveau jeton d'accès sans redemander le mot de passe). Ce découplage limite la fenêtre d'exploitation d'un jeton d'accès volé (il expire vite) tout en évitant de forcer l'utilisateur à se reconnecter en permanence.

2.6.3 Autorisation par rôle (RBAC)



Le problème. Une fois l'identité prouvée, il faut décider ce que cet utilisateur a le droit de faire : un sportif ne doit pas pouvoir supprimer la séance d'un autre coach.

La théorie : le contrôle d'accès basé sur les rôles (*Role-Based Access Control*). Plutôt que d'attribuer des permissions individuellement à chaque utilisateur, on définit un petit

nombre de **rôles** (ici : sportif, coach, admin), chacun associé à un ensemble de permissions, et on assigne un rôle à chaque utilisateur. C'est un compromis entre la simplicité (peu de rôles à gérer) et la granularité (un rôle regroupe des permissions cohérentes). SportPlan combine RBAC avec une vérification de **propriété** plus fine dans certains cas (un coach ne peut modifier que les séances dont il est l'animateur : `session.coach_id == user.id`), ce qui est un contrôle d'accès *contextuel*, complémentaire au RBAC pur.



Un principe de sécurité essentiel appliqué ici : ne jamais faire confiance au client. Le frontend cache des boutons ou redirige selon le rôle perçu, uniquement pour l'ergonomie — mais **chaque route sensible du backend revérifie indépendamment le rôle et la propriété**. Un attaquant qui modifierait le JavaScript exécuté dans son navigateur, ou qui appellerait l'API directement sans passer par l'interface, se heurterait aux mêmes vérifications côté serveur. C'est le principe de **défense en profondeur** : la sécurité ne repose jamais sur une seule couche.

2.6.4 CORS



Le problème. Pour des raisons de sécurité, un navigateur applique par défaut la *same-origin policy* : une page chargée depuis `http://localhost:4200` ne peut pas, par défaut, lire la réponse d'une requête envoyée vers une autre origine comme `http://localhost:8000` (port différent = origine différente).



La théorie : Cross-Origin Resource Sharing. CORS est un mécanisme par lequel le serveur déclare explicitement, via des en-têtes HTTP (`Access-Control-Allow-Origin...`), quelles origines sont autorisées à consommer son API. Sans cette déclaration côté FastAPI, le navigateur bloquerait toutes les requêtes du frontend Angular vers le backend, même si le serveur y répondait correctement — le blocage a lieu côté navigateur, en protection de l'utilisateur.

2.6.5 Limitation de débit (*rate limiting*)



Le problème. Sans limite, un endpoint sensible comme la connexion peut être soumis à une attaque par force brute (essayer des milliers de mots de passe automatiquement) ou l'API entière à un déni de service par saturation.



La théorie. Le *rate limiting* borne le nombre de requêtes qu'une même source (typiquement identifiée par adresse IP) peut effectuer sur une fenêtre de temps donnée, au-delà de laquelle les requêtes supplémentaires sont rejetées (HTTP 429).

2.7 Programmation réactive côté frontend (RxJS)



Le problème général. Une interface utilisateur moderne doit réagir à des événements asynchrones multiples et souvent combinés : une requête HTTP qui répond, un clic utilisateur, un minuteur qui s'écoule. Enchaîner ces événements avec des callbacks imbriqués (le « *callback hell* ») rend le code difficile à suivre et à composer.

La théorie : les flux observables (*Observables*). La programmation réactive modélise chaque source de données asynchrone comme un **flux** (*stream*) de valeurs dans le temps, auquel on peut s'abonner (*subscribe*) et sur lequel on peut appliquer des **opérateurs** qui le transforment en un nouveau flux, de façon déclarative et composable (à la manière des méthodes *map/filter* sur un tableau, mais étalées dans le temps). RxJS est la bibliothèque qui implémente ce paradigme pour Angular ; `HttpClient` renvoie systématiquement un *Observable*.

Opérateurs effectivement utilisés dans SportPlan, chacun résolvant un problème de composition précis :

- `switchMap` : enchaîne un second flux asynchrone à partir de la valeur du premier (ex. « une fois l'utilisateur courant connu, charger ses données »), en annulant automatiquement une requête précédente encore en cours si une nouvelle valeur arrive entre-temps.
- `combineLatest` : combine plusieurs flux indépendants et réémet dès que l'un d'eux change, avec la dernière valeur connue de chacun (utilisé par exemple pour croiser la liste des participations avec celle des séances).
- `forkJoin` : attend que plusieurs requêtes ponctuelles se terminent toutes, puis émet une seule fois leurs résultats combinés (utilisé pour des opérations « tout ou rien », comme enregistrer plusieurs exercices en parallèle).
- `shareReplay` : mémorise la dernière valeur émise et la rediffuse immédiatement à tout nouvel abonné, évitant qu'un même appel HTTP (par exemple `/users/me`) ne soit déclenché deux fois si deux parties du code s'abonnent au même flux.
- `BehaviorSubject` : un flux qui retient toujours sa dernière valeur et permet de la redéclencher manuellement (`.next()`) — utilisé comme déclencheur de rafraîchissement après une mutation (ex. après une désinscription, sur la page Participations).

2.8 Composants standalone, injection de dépendances et détection de changement (Angular)

Composants standalone. Historiquement, Angular imposait de déclarer chaque composant dans un `NgModule`. Depuis les versions récentes (dont Angular 22 utilisé ici), un composant peut se déclarer `standalone: true` et importer directement les autres briques dont il a besoin (`imports: [...]` dans le décorateur `@Component`), ce qui simplifie l'arborescence de dépendances et rend chaque composant plus explicite sur ce qu'il utilise réellement.

Injection de dépendances (DI). Plutôt que chaque classe crée elle-même les objets dont elle dépend (un composant qui ferait `new HttpClient()` lui-même), le framework **injecte** ces dépendances de l'extérieur — ici via la fonction `inject()` appelée dans le corps de la

classe. Un service marqué `@Injectable({ providedIn: 'root' })` devient un **singleton** partagé par toute l'application : tous les composants qui l'injectent reçoivent la même instance, ce qui permet de partager un état (par exemple le thème clair/sombre) sans mécanisme de state management lourd. La DI facilite aussi les tests, puisqu'on peut substituer une dépendance réelle par une version simulée (*mock*).

Détection de changement. Angular doit savoir quand rafraîchir le DOM après un changement de donnée. Le projet utilise un opérateur RxJS interne au projet, `markForCheck` (`core/mark-for-check.operator.ts`), appliqué systématiquement sur les abonnements aux appels HTTP : il force Angular à révérifier le composant après qu'une valeur asynchrone soit arrivée en dehors du cycle de détection automatique habituel — un signe que l'application tire parti d'une stratégie de détection plus fine que le mode par défaut (cohérent avec les évolutions récentes d'Angular vers un mode *zoneless*, moins dépendant de la détection automatique globale).

Gardes de route (*route guards*). `core/auth.guard.ts` définit des fonctions (`authGuard`, `coachGuard`, `adminGuard`) exécutées par le routeur Angular *avant* d'activer une route, qui peuvent bloquer la navigation ou rediriger si l'utilisateur n'est pas authentifié ou n'a pas le rôle requis. Comme expliqué en §2.6.3, ces gardes ne sont qu'un confort d'expérience utilisateur : la vérification qui compte réellement est celle refaite côté serveur sur chaque route.

Intercepteur HTTP. `features/auth/auth.interceptor.ts` s'insère dans le pipeline de chaque requête HTTP sortante pour y ajouter automatiquement l'en-tête `Authorization: Bearer <token>`, évitant de dupliquer cette logique dans chaque service. C'est une application du patron **Chain of Responsibility** : chaque intercepteur peut inspecter/modifier la requête avant de la transmettre au suivant dans la chaîne.

2.9 Design d'interface : variables CSS et thématisation

Le thème clair/sombre repose sur des **variables CSS personnalisées** (custom properties, `--accent-primary`, `--bg-surface`...) définies une seule fois dans `:root` et redéfinies sous un sélecteur `html[data-theme='dark']`. Toute règle de style qui référence `var(--accent-primary)` change donc de valeur instantanément selon l'attribut `data-theme` posé sur le document par `ThemeService`, sans dupliquer la moindre feuille de style entre les deux thèmes — une application directe du principe **DRY** (*Don't Repeat Yourself*) au design visuel.

2.10 Tests automatisés et pyramide des tests



La théorie. La *pyramide des tests* recommande une base large de tests **unitaires** (rapides, isolés, nombreux), un niveau intermédiaire de tests **d'intégration**, et un sommet réduit de tests **de bout en bout** (lents, coûteux à maintenir). Un test unitaire vérifie une seule unité de code (une fonction, une classe) indépendamment du reste du système.

Application dans SportPlan. Le projet se concentre délibérément sur les tests unitaires les plus critiques et les moins coûteux à écrire : la validation Pydantic (`test_schemas.py` —

vérifie que les contraintes métier, comme un objectif à valeur cible strictement positive, sont bien rejetées), la sécurité (`test_security.py` — hachage/vérification de mot de passe, création et expiration de jetons JWT), et une fonction de service pure (`test_time.py`). Ce choix illustre un principe pragmatique de test : prioriser les zones à plus haut risque (sécurité, validation des règles métier) plutôt que de viser une couverture exhaustive dès le départ.

2.11 Principes transverses de génie logiciel illustrés par le projet

Pour clore ce chapitre théorique, quelques principes généraux de génie logiciel, cités dans la littérature académique, et concrètement observables dans le code de SportPlan :

- **Séparation des préoccupations (*Separation of Concerns*)** : chaque fichier a une seule raison de changer (un `router` change si une règle d'exposition HTTP change, un `model` change si le schéma de la base change). C'est aussi une lecture du **principe de responsabilité unique** (le S du sigle SOLID).
- **Ne jamais faire confiance à l'entrée (*validate at the boundary*)** : la validation Pydantic se fait à la frontière de l'application (dès la réception de la requête), avant toute logique métier — un principe de sécurité défensive.
- **Fail fast** : `Settings` (`config.py`) n'a pas de valeur par défaut pour `secret_key/database_url` : l'application refuse de démarrer plutôt que de tourner silencieusement dans une configuration incomplète ou non sécurisée.
- **DRY (*Don't Repeat Yourself*)** : la thématisation CSS (§2.9), les services Angular partagés par plusieurs composants, ou l'opérateur `markForCheck` mutualisé en sont des illustrations. À l'inverse, la revue de code (chapitre 7) relève aussi des cas où ce principe n'est pas respecté (logique de calcul d'assiduité dupliquée entre deux composants, filtrage par rôle réécrit dans quatre routeurs) — un bon dossier technique doit signaler les deux, pas seulement les réussites.
- **YAGNI (*You Aren't Gonna Need It*)** : plusieurs ressources n'exposent que les opérations réellement utilisées par l'interface (pas de `PATCH` sur un programme, pas de `DELETE` sur un record personnel) plutôt qu'un CRUD complet systématique — un choix raisonnable pour un projet de cette taille, tant que l'absence est un choix assumé et non un oubli.

3. Stack technique détaillée

03

3.1 Backend (Python)

Bibliothèque	Version épinglée	Rôle
fastapi	0.115.6	Framework web ASGI : routage, validation automatique, documentation OpenAPI générée
uvicorn[standard]	0.34.0	Serveur ASGI qui exécute l'application FastAPI
sqlalchemy	2.0.51	ORM et couche d'accès base de données (API « 2.0 » typée)
pyodbc	5.3.0	Pilote bas niveau de connexion à SQL Server, utilisé par SQLAlchemy
pydantic-settings	2.7.1	Chargement typé de la configuration depuis les variables d'environnement / .env
python-dotenv	1.0.1	Lecture du fichier .env en développement
python-jose[cryptography]	3.3.0	Encodage/décodage et signature des jetons JWT
argon2-cffi	23.1.0	Hachage des mots de passe (algorithme Argon2)
python-multipart	0.0.20	Support des formulaires application/x-www-form-urlencoded, requis par OAuth2PasswordRequestForm
email-validator	2.2.0	Validation du format des adresses courriel par Pydantic (EmailStr)
pytest	9.1.1 (dev)	Exécution des tests unitaires

FastAPI a été choisi pour trois raisons théoriques concrètes : (1) il est construit sur Pydantic, ce qui rend la validation des données (§2.5) native plutôt que rajoutée après coup ; (2) il est asynchrone par conception (ASGI), ce qui autorise de servir de nombreuses requêtes simultanées avec un seul processus sans bloquer sur les entrées/sorties ; (3) il génère automatiquement une documentation interactive (OpenAPI/Swagger) à partir des types Python et des schémas déclarés, sans documentation à maintenir manuellement.

3.2 Frontend (Angular)

Bibliothèque	Version	Rôle
@angular/core (+ modules associés)	^22.0.0	Framework SPA : composants, routage, formulaires, injection de dépendances
@angular/material	^22.0.0	Bibliothèque de composants UI (cartes, boutons, champs de formulaire...) suivant Material Design
chart.js	^4.5.0	Génération des graphiques (évolution des performances, taux de présence)
rxjs	~7.8.0	Programmation réactive (Observables, opérateurs — voir §2.7)
zone.js	~0.15.0	Historiquement responsable de la détection de changement automatique d'Angular (voir la nuance en §2.8 sur le mode zoneless)
typescript	~6.0.0	Sur-ensemble typé de JavaScript, compilé vers JavaScript standard

Bibliothèque	Version	Rôle
vitest	^4.1.11 (dev)	Exécution des tests unitaires frontend

Angular a été choisi pour sa structure imposée (composants, services, modules ou, ici, composants *standalone*) qui convient à une application « riche » comportant de nombreux écrans interconnectés, et pour son intégration native avec RxJS et TypeScript (typage statique, détecté dès la compilation plutôt qu'à l'exécution).

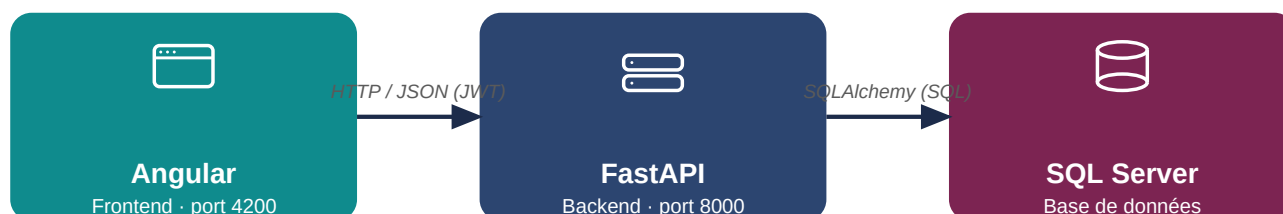
3.3 Base de données et sécurité

- **SQL Server**, avec une instance nommée en développement local — moteur relationnel respectant les propriétés **ACID** (Atomicité, Cohérence, Isolation, Durabilité) des transactions, essentielles ici puisque plusieurs écritures liées (ex. créer une séance et notifier ses inscrits) doivent réussir ou échouer ensemble.
- **Argon2** pour le hachage des mots de passe (§2.6.1).
- **JWT (JSON Web Token)** pour l'authentification sans état (§2.6.2).
- **CORS** configuré explicitement (§2.6.4).

4. Architecture générale du projet

04

4.1 Vue d'ensemble et flux d'une requête



Le trajet complet d'une action utilisateur illustre l'empilement de toutes les couches décrites au chapitre 2 :

1. L'utilisateur clique sur un bouton dans un **composant** Angular.
2. Le composant appelle une méthode d'un **service** Angular, qui construit une requête HTTP via `HttpClient`.
3. L'**intercepteur** (`authInterceptor`) ajoute automatiquement le jeton d'accès à l'en-tête `Authorization`.
4. La requête traverse le réseau (localhost en développement) et atteint FastAPI, où le middleware **CORS** valide l'origine.
5. FastAPI route la requête vers la fonction du **routeur** concerné, dont la signature déclare les **dépendances** nécessaires (`Depends(get_db)`, `Depends(get_current_user)` ou `Depends(require_roles(...))`) — chacune est résolue avant l'exécution de la fonction.
6. Le corps de la requête est validé contre le **schéma** Pydantic déclaré ; toute anomalie interrompt le traitement avec une erreur 422 avant même d'atteindre le code métier.
7. Le routeur applique les règles métier (autorisation fine, contraintes temporelles...) et manipule les **modèles** SQLAlchemy via la **session** de base de données.
8. La session traduit ces manipulations en une transaction SQL envoyée à SQL Server.

9. Le routeur renvoie un objet, sérialisé automatiquement selon le schéma Pydantic déclaré en `response_model`.
10. Côté frontend, le service Angular reçoit la réponse (un `Observable` qui émet une valeur), et le composant met à jour son état, ce qui déclenche le rafraîchissement de la vue.

4.2 Organisation des dossiers

L'arborescence suit deux logiques différentes, chacune adaptée à son contexte (détaillé au chapitre 7) :

- **Backend — organisation par couche technique** (`models/`, `schemas/`, `routers/`, `services/`) : cohérent avec une API dont la complexité provient surtout de la manipulation de données structurées.
- **Frontend — organisation par fonctionnalité** (*feature-based*, `features/sessions/`, `features/goals/...`) : chaque dossier regroupe le composant d'écran et son service dédié. Ce choix évite d'avoir, comme dans une organisation par type de fichier, un dossier `components/` contenant des dizaines de composants sans rapport entre eux ; il facilite aussi la suppression ou l'extraction d'une fonctionnalité complète.

4.3 Alias d'import (frontend)

Trois alias TypeScript (`@core/*`, `@shared/*`, `@features/*`, définis dans `tsconfig.json`) évitent les chemins relatifs profonds (`../../../../shared/services/toast.service`) : un import vers une autre fonctionnalité ou du code transverse utilise l'alias, un import au sein d'un même dossier reste relatif (`./goal.service`).

5. Modèle de données

05

User est l'entité pivot du schéma relationnel : tout le reste du modèle en dépend directement ou indirectement, ce qui est cohérent avec une application dont la sécurité repose sur l'identité et le rôle de chaque utilisateur (§2.6.3).

5.1 Tableau des entités

Table	Colonnes principales	Clés étrangères	Relations ORM (relationship)
users	id, email (unique), full_name, hashed_password, role (enum SQL sportif/coach/admin), is_active	—	sessions (1-N vers Session), participations (1-N vers Participation)
sport_sessions	id, title, description, starts_at, duration_minutes, capacity, coach_id	coach_id -> users.id	coach (N-1), exercises (1-N, cascade suppression), participations (1-N, cascade suppression)
participations	id, user_id, session_id, status (enum inscrit/present/absent)	user_id -> users.id, session_id -> sport_sessions.id	user, session
performances	id, user_id, session_id, score, notes, recorded_at	user_id -> users.id, session_id -> sport_sessions.id	user, session (sans back_populates)
exercises	id, session_id, name, sets, repetitions, rest_seconds	session_id -> sport_sessions.id	session
goals	id, user_id, title, metric, target_value, current_value, unit, due_date	user_id -> users.id (cascade DB)	aucune
personal_records	id, user_id, exercise_name (texte libre), value, unit, achieved_at	user_id -> users.id (cascade DB)	aucune
workout_programs	id, user_id, name, weeks, sessions (JSON)	user_id -> users.id (cascade DB)	aucune
training_journals	id, user_id, session_id, fatigue, mood, pain, coach_comment	user_id -> users.id, session_id -> sport_sessions.id (cascade DB), unique(user_id,session_id)	aucune
notifications	id, user_id, title, message, kind, is_read	user_id -> users.id (cascade DB)	aucune
refresh_tokens	id, token_hash (unique), user_id, expires_at, revoked_at	user_id -> users.id (cascade DB)	user

5.2 Lecture du graphe relationnel

User (1) — (N) Session (*coach*) : un utilisateur ayant le rôle coach anime plusieurs séances. User (N) — Participation — (N) Session : Participation est la table d'association enrichie (statut de présence) qui matérialise la relation many-to-many entre sportifs et séances (§2.4). Session (1) — (N) Exercise, Performance, TrainingJournal : chaque séance regroupe ses propres exercices, les performances qui y ont été réalisées, et les entrées de journal qui s'y rapportent. User (1) — (N) Goal, PersonalRecord, WorkoutProgram, Notification, RefreshToken : ressources strictement personnelles.

Deux stratégies de suppression en cascade coexistent dans le projet : côté ORM (`cascade="all, delete-orphan"` sur `Session.exercises` et `Session.participations` — SQLAlchemy émet lui-même les DELETE enfants avant de supprimer le parent) et côté base de données (`onDelete="CASCADE"` sur les clés étrangères de `Goal`, `PersonalRecord`, `WorkoutProgram`, `TrainingJournal`, `Notification`, `RefreshToken` — c'est SQL Server qui applique la suppression en cascade au niveau du moteur). Les deux approches sont valides individuellement, mais leur coexistence dans le même projet est un choix non homogène à noter dans une revue.

5.3 Écarts de contrainte entre schéma Pydantic et modèle SQLAlchemy

La revue de code (chapitre 7) relève plusieurs cas où la contrainte déclarée dans le schéma d'entrée (Pydantic, appliquée à la création via l'API) est plus stricte que la colonne correspondante en base (SQLAlchemy) :

- `Exercise.sets` est obligatoire et borné (`Field(gt=0, le=30)`) dans `ExerciseCreate`, mais la colonne `exercises.sets` est `nullable=True` — une ligne pourrait exister en base avec `sets = NULL` si elle était créée par un autre chemin que l'API (ex. script de seed).
- `role` (users) et `status` (participations) sont contraints par un Enum **SQL**, mais acceptés comme simples chaînes libres côté schéma Pydantic (`UserCreate.role: str`, `ParticipationUpdate.status: str`) : une valeur invalide serait rejetée seulement au moment de l'insertion en base (erreur SQL peu explicite), plutôt qu'immédiatement par une erreur 422 lisible.

Ce type d'écart illustre concrètement pourquoi la validation doit idéalement être **cohérente à toutes les couches** (défense en profondeur, §2.6.3), et pas seulement présente à une seule.

Ce chapitre relie la théorie du §2.6 à son implémentation réelle, observée dans `security.py`, `dependencies.py`, `rate_limit.py` et `routers/auth.py`.

6.1 Cycle de vie des jetons

1. **Connexion** (POST `/auth/login`) : le mot de passe est vérifié via Argon2 (`verify_password`) ; en cas de succès, un jeton d'accès (`create_access_token`, `ACCESS_TOKEN_EXPIRE_MINUTES` = 30 par défaut) et un jeton de rafraîchissement (`create_refresh_token`, `REFRESH_TOKEN_EXPIRE_DAYS` = 7 par défaut) sont émis. Chaque jeton encode `sub` (l'identifiant utilisateur), `type` ("access" ou "refresh") et `exp` (expiration), signés en **HS256** avec la clé secrète du serveur.
2. **Persistance du refresh token** : plutôt que de stocker le jeton en clair, seul son **empreinte SHA-256** (`_token_hash`) est enregistrée dans la table `refresh_tokens`, avec sa date d'expiration. Une fuite de la base ne permettrait donc pas de rejouer un jeton de rafraîchissement directement.
3. **Requêtes protégées** : `get_current_user` (dépendance FastAPI) décode le jeton d'accès, vérifie `type == "access"`, et charge l'utilisateur correspondant ; toute anomalie renvoie une erreur 401 uniforme ("Token *invalid*"), sans détailler la cause exacte à l'appelant (évite de renseigner un attaquant sur la nature précise de l'échec).
4. **Rafraîchissement** (POST `/auth/refresh`) : le jeton de rafraîchissement est revérifié (`type`, présence en base via son hash, `is_valid` — ni révoqué ni expiré), puis **immédiatement révoqué** et remplacé par une nouvelle paire de jetons (**rotation**). Cette rotation limite la fenêtre d'usage d'un jeton de rafraîchissement volé : dès qu'il est utilisé une fois (par le légitime détenteur ou un attaquant), il devient inutilisable pour la suite.
5. **Déconnexion** (POST `/auth/logout`) : révoque explicitement le jeton de rafraîchissement fourni (opération idempotente).

6.2 Autorisation en profondeur

Deux mécanismes se combinent systématiquement dans les routeurs sensibles : `Depends(require_roles(...))` (contrôle **RBAC**, §2.6.3) pour les actions réservées à un rôle, et une vérification de **propriété** explicite en code (`if item.coach_id != user.id` :

`raise HTTPException(403, ...))` pour les actions qu'un rôle ne peut effectuer que sur ses propres ressources. Le même schéma de filtrage par rôle (admin voit tout, coach voit les ressources liées à ses séances via une jointure sur `SportSession.coach_id`, sportif voit les siennes) est répété dans `participations.py`, `performances.py`, `journal.py` et `statistics.py` — une factorisation dans une fonction utilitaire commune serait une amélioration naturelle (voir chapitre 10).

6.3 Limitation de débit sur l'authentification

`login_rate_limit` (`rate_limit.py`) borne les tentatives de connexion et d'inscription à 10 par minute et par adresse IP, stockées en mémoire du processus (une deque d'horodatages par IP). Cette implémentation, volontairement simple, a deux limites explicitement documentées dans le code : elle ne survit pas à un redémarrage du serveur, et elle ne serait pas partagée entre plusieurs instances du backend derrière un répartiteur de charge (il faudrait alors un magasin partagé, par exemple Redis).

6.4 CORS

Le middleware CORS de FastAPI (`main.py`) autorise explicitement les origines listées dans `ALLOWED_ORIGINS` (par défaut `http://localhost:4200` et `http://127.0.0.1:4200`, les ports du serveur de développement Angular), avec `allow_credentials=True`. Les méthodes et en-têtes sont en revanche laissés ouverts (`["*"]`) : seul le filtrage par origine est restrictif.

7. Revue de code fichier par fichier

07

Cette section documente chaque fichier source du projet. Le format est volontairement homogène : **Rôle**, **Contenu clé** (classes, fonctions, endpoints), **Points notables** (patterns illustrés et remarques de qualité). Les chemins sont donnés relativement à la racine du dépôt.

7.1 Backend (backend/app/)

7.1.1 Point d'entrée, configuration et sécurité transverse

`app/main.py` — *Rôle* : point d'entrée de l'application FastAPI. *Contenu clé* : instancie FastAPI, configure le middleware CORS à partir de `get_settings().allowed_origins_list`, enregistre les 11 routeurs du projet (`auth`, `users`, `sessions`, `exercices`, `participations`, `performances`, `statistics`, `goals`, `programs`, `notifications`, `journal`), définit un gestionnaire `lifespan` qui appelle `Base.metadata.create_all(bind=engine)` au démarrage, et expose `GET /health` (sonde de disponibilité, sans authentification). *Points notables* : tous les modèles SQLAlchemy sont importés explicitement (avec `# noqa: F401`) uniquement pour forcer leur enregistrement dans les métadonnées avant `create_all()` — un modèle oublié dans cette liste ne serait jamais matérialisé en base ; le schéma est créé par introspection des modèles plutôt que par des migrations versionnées (pas d'Alembic), ce qui est une simplification acceptable pour un projet scolaire mais deviendrait un point de vigilance en production (une évolution de schéma sur une base déjà peuplée n'est pas gérée automatiquement).

PY `backend/app/main.py` (lignes 1-fin)

```
"""Point d'entrée de l'application FastAPI : création de l'app, configuration du
    CORS,
    enregistrement des routeurs et création automatique des tables au démarrage.
"""

from contextlib import asynccontextmanager

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from .config import get_settings
from .database import Base, engine
# Les modèles sont importés ici (même sans usage direct) pour que SQLAlchemy
# les enregistre dans Base.metadata avant l'appel à create_all().
from .models import Exercise, Goal, Notification, Participation, Performance,
    PersonalRecord, Session, TrainingJournal, User, WorkoutProgram # noqa: F401
from .routers import auth, exercices, goals, journal, notifications,
    participations, performances, programs, sessions, statistics, users
```

```

@asynccontextmanager
async def lifespan(_: FastAPI):
    """Cycle de vie de l'application : crée les tables manquantes en base au
    démarrage."""
    Base.metadata.create_all(bind=engine)
    yield

app = FastAPI(title="Sports Sessions API", version="1.0.0", lifespan=lifespan)
app.add_middleware(
    CORSMiddleware,
    allow_origins=get_settings().allowed_origins_list,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
app.include_router(auth.router)
app.include_router(users.router)
app.include_router(sessions.router)
app.include_router(exercises.router)
app.include_router(participations.router)
app.include_router(performances.router)
app.include_router(statistics.router)
app.include_router(goals.router)
app.include_router(programs.router)
app.include_router(notifications.router)
app.include_router(journal.router)

@app.get("/health", tags=["health"])
def health():
    """Point de contrôle de santé (GET /health) : confirme que l'API répond."""
    return {"status": "ok"}

```

app/config.py — *Rôle* : centralise la configuration lue depuis l'environnement/.env via pydantic-settings. *Contenu clé* : Settings(BaseSettings) avec database_url/secret_key obligatoires (aucune valeur par défaut), access_token_expire_minutes=30, refresh_token_expire_days=7, allowed_origins (CSV converti en liste par la propriété allowed_origins_list) ; get_settings() décoré @lru_cache pour ne construire l'objet qu'une seule fois. *Points notables* : illustre le principe *fail fast* (§2.11) — sans SECRET_KEY définie, l'application refuse de démarrer plutôt que d'utiliser une valeur par défaut non sécurisée.

PY backend/app/config.py (lignes 1-fin)

```

"""Configuration de l'application, chargée depuis les variables d'environnement ou
un fichier .env."""

from functools import lru_cache

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    """Paramètres globaux de l'application (base de données, JWT, CORS)."""

    database_url: str
    secret_key: str
    access_token_expire_minutes: int = 30
    refresh_token_expire_days: int = 7
    allowed_origins: str = "http://localhost:4200,http://127.0.0.1:4200"

    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8",
        extra="ignore")

    @property
    def allowed_origins_list(self) -> list[str]:
        """Convertit la liste d'origines CORS (chaîne séparée par des virgules) en
        liste de chaînes."""
        return [origin.strip() for origin in self.allowed_origins.split(",") if
            origin.strip()]

    @lru_cache
    def get_settings() -> Settings:
        """Retourne l'instance unique des paramètres (mise en cache pour éviter de
        relire l'environnement)."""
        return Settings()

```

app/database.py — *Rôle* : configure SQLAlchemy et fournit une session par requête. *Contenu clé* : Base(DeclarativeBase) (classe de base ORM) ; engine = create_engine(..., pool_pre_ping=True, future=True) ; SessionLocal = sessionmaker(...); get_db() — générateur qui ouvre une session, la cède (yield) au routeur, puis la ferme dans un bloc finally (garantie de fermeture même en cas d'exception). *Points notables* : pool_pre_ping=True évite les erreurs dues à une connexion SQL Server devenue obsolète (testée avant chaque emprunt au pool) ; pattern canonique de dépendance FastAPI pour l'accès base de données.

PY backend/app/database.py (lignes 1-fin)

```

"""Configuration de l'accès à la base de données : moteur SQLAlchemy, fabrique de
sessions
et classe de base déclarative utilisée par tous les modèles ORM.
"""

from collections.abc import Generator

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

from .config import get_settings

class Base(DeclarativeBase):
    """Classe de base déclarative SQLAlchemy dont héritent tous les modèles ORM."""
    pass
engine = create_engine(get_settings().database_url, pool_pre_ping=True,
                        future=True)
SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

def get_db() -> Generator[Session, None, None]:
    """Dépendance FastAPI fournissant une session de base de données par requête,
    fermée automatiquement à la fin (même en cas d'exception).
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

app/security.py — *Rôle* : hachage des mots de passe et gestion des JWT (détaillé au §6.1). *Contenu clé* : hash_password/verify_password (Argon2, via argon2_cffi.PasswordHasher); create_token (générique), create_access_token, create_refresh_token (durées différenciées) ; decode_token (vérifie signature et expiration, lève JWTErrors). *Points notables* : verify_password capture les exceptions de la bibliothèque et retourne simplement False, plutôt que de laisser remonter une exception — évite qu'un mot de passe malformé casse le flux de connexion avec une erreur 500.

PY backend/app/security.py (lignes 1-fin)

```

"""Fonctions de sécurité : hachage/vérification des mots de passe (Argon2) et
création/décodage des jetons JWT d'accès et de rafraîchissement.
"""

from datetime import datetime, timedelta, timezone

from jose import JWTError, jwt
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError, VerificationError

from .config import get_settings

ALGORITHM = "HS256"
password_hasher = PasswordHasher()

def verify_password(plain: str, hashed: str) -> bool:
    """Vérifie qu'un mot de passe en clair correspond au hachage Argon2 stocké."""
    try:
        return password_hasher.verify(hashed, plain)
    except (VerifyMismatchError, VerificationError):
        return False

def hash_password(password: str) -> str:
    """Calcule le hachage Argon2 d'un mot de passe en clair, à stocker en base."""
    return password_hasher.hash(password)

def create_token(subject: str, token_type: str, expires_delta: timedelta) -> str:
    """Crée un JWT signé pour `subject` (identifiant utilisateur), avec un type
    ("access" ou "refresh") et une durée de validité donnée.
    """
    now = datetime.now(timezone.utc)
    payload = {"sub": subject, "type": token_type, "iat": now, "exp": now +
        expires_delta}
    return jwt.encode(payload, get_settings().secret_key, algorithm=ALGORITHM)

def create_access_token(subject: str) -> str:
    """Crée un jeton d'accès de courte durée (voir access_token_expire_minutes)."""
    return create_token(subject, "access",
        timedelta(minutes=get_settings().access_token_expire_minutes))

def create_refresh_token(subject: str) -> str:
    """Crée un jeton de rafraîchissement de longue durée (voir
    refresh_token_expire_days)."""
    return create_token(subject, "refresh",
        timedelta(days=get_settings().refresh_token_expire_days))

def decode_token(token: str) -> dict:
    """Décode et vérifie la signature d'un JWT, retourne son contenu (payload).
    Lève une JWTError si le token est invalide, expiré ou mal signé.
    """
    return jwt.decode(token, get_settings().secret_key, algorithms=[ALGORITHM])

```

app/dependencies.py — Rôle : dépendances FastAPI réutilisables d'authentification/autorisation. Contenu clé : oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login") ; get_current_user(token, db) (décode le jeton, vérifie type == "access", charge l'utilisateur, vérifie is_active, lève 401 sinon) ; require_roles(*roles) (fabrique une dépendance qui lève 403 si user.role n'est pas dans la liste). Points notables : toutes les causes d'échec de décodage (JWTError, ValueError, TypeError) sont ramenées à la même erreur 401 générique — bonne pratique pour ne pas renseigner un attaquant sur la nature exacte de l'échec ; require_roles est composable (require_roles("coach", "admin")), utilisé de façon cohérente dans les routeurs qui en ont besoin.

PY backend/app/dependencies.py (lignes 1-fin)

```
"""Dépendances FastAPI réutilisables pour l'authentification et le contrôle d'accès
par rôle (utilisées via Depends() dans les routeurs).
"""

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError
from sqlalchemy.orm import Session

from .database import get_db
from .models.user import User
from .security import decode_token

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

def get_current_user(token: str = Depends(oauth2_scheme), db: Session =
    Depends(get_db)) -> User:
    """Décode le token JWT d'accès fourni dans l'en-tête Authorization et retourne
    l'utilisateur actif correspondant. Lève une 401 si le token est invalide,
    n'est pas de type "access", ou si l'utilisateur est introuvable/inactif.
    """
    credentials_error = HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Token invalide", headers={"WWW-Authenticate": "Bearer"})
    try:
        payload = decode_token(token)
        if payload.get("type") != "access" or not payload.get("sub"):
            raise credentials_error
        user = db.get(User, int(payload["sub"]))
    except (JWTError, ValueError, TypeError):
        raise credentials_error
    if user is None or not user.is_active:
        raise credentials_error
    return user
```

```
def require_roles(*roles: str):
    """Fabrique une dépendance FastAPI qui restreint l'accès à un endpoint aux
    utilisateurs dont le rôle figure dans `roles` (ex: require_roles("coach")).
    Lève une 403 si le rôle de l'utilisateur courant n'est pas autorisé.
    """

    def checker(user: User = Depends(get_current_user)) -> User:
        if user.role not in roles:
            raise HTTPException(status_code=403, detail="Permissions
            insuffisantes")
        return user

    return checker
```

`app/rate_limit.py` — *Rôle* : limitation de débit sur les endpoints d'authentification (détaillé au §6.3). *Contenu clé* : dictionnaire en mémoire `_attempts: dict[str, deque[datetime]]` indexé par adresse IP ; `login_rate_limit(request)` purge les tentatives de plus d'une minute et lève 429 au-delà de 10 tentatives/minute/IP. *Points notables* : limite documentée dans le code lui-même — stockage mono-processus, non partagé entre plusieurs instances ; clé de limitation basée uniquement sur l'IP source (`request.client.host`), sans tenir compte d'un éventuel X-Forwarded-For derrière un reverse proxy.

PY `backend/app/rate_limit.py` (lignes 1-fin)

```
"""Limitation de débit (rate limiting) simple en mémoire pour protéger l'endpoint
de connexion
contre les tentatives de force brute.
"""

from collections import defaultdict, deque
from datetime import datetime, timedelta, timezone

from fastapi import HTTPException, Request, status

# Stockage en mémoire (par processus) des horodatages de tentatives par adresse IP.
# Non partagé entre plusieurs instances de l'API : suffisant pour ce projet mais
# ne résiste pas à un redémarrage ou à un déploiement multi-instance.
_attempts: dict[str, deque[datetime]] = defaultdict(deque)

def login_rate_limit(request: Request) -> None:
    """Dépendance FastAPI limitant à 10 tentatives de connexion par minute et par
    IP.
    Lève une 429 (Too Many Requests) au-delà de cette limite.
    """
    now = datetime.now(timezone.utc)
    key = request.client.host if request.client else "unknown"
    attempts = _attempts[key]
    while attempts and attempts[0] < now - timedelta(minutes=1):
        attempts.popleft()
    if len(attempts) >= 10:
        raise HTTPException(status.HTTP_429_TOO_MANY_REQUESTS, "Trop de
        tentatives. Réessayez plus tard.")
    attempts.append(now)
```

app/services/time.py — *Rôle* : utilitaire de comparaison temporelle partagé. *Contenu clé* : `is_past(value: datetime) -> bool`, normalise la valeur en UTC si elle est naïve avant de la comparer à `datetime.now(timezone.utc)`. *Points notables* : utilisé dans `sessions.py`, `participations.py` et `journal.py` pour interdire toute action sur une séance déjà passée — bon exemple de logique métier pure, indépendante de FastAPI, extraite dans `services/` (cf. §2.3) et donc testable isolément (`test_time.py`).

PY backend/app/services/time.py (lignes 1-fin)

```
"""Fonctions utilitaires de manipulation des dates/heures."""

from datetime import datetime, timezone

def is_past(value: datetime) -> bool:
    """Indique si une date/heure donnée est déjà passée par rapport à maintenant (UTC).
    Les valeurs naïves (sans fuseau) sont supposées être en UTC.
    """
    normalized = value if value.tzinfo else value.replace(tzinfo=timezone.utc)
    return normalized <= datetime.now(timezone.utc)
```

7.1.2 Modèles SQLAlchemy (app/models/)

models/__init__.py — Ré-exporte les 11 classes de modèles via `__all__`, garantissant qu'elles sont toutes enregistrées auprès de Base avant `create_all()`.

models/user.py — Table users. Colonnes : id, email (unique, indexé), full_name, hashed_password, role (Enum SQL "coach"|"sportif"|"admin", défaut "sportif"), is_active (défaut True). Relations : sessions (1-N vers Session), participations (1-N vers Participation). *Notable* : le rôle est contraint au niveau base par un type énuméré SQL — une valeur invalide est rejetée par le moteur, pas seulement par l'application.

PY backend/app/models/user.py (lignes 15-27)

```
class User(Base):
    """Compte utilisateur : identité, mot de passe haché, rôle et statut
    actif/inactif."""

    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    full_name: Mapped[str] = mapped_column(Unicode(150))
    hashed_password: Mapped[str] = mapped_column(String(255))
    role: Mapped[str] = mapped_column(Enum("coach", "sportif", "admin",
        name="user_roles"), default="sportif")
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    sessions: Mapped[list["Session"]] = relationship(back_populates="coach")
    participations: Mapped[list["Participation"]] =
        relationship(back_populates="user")
```

models/session.py — Table sport_sessions. Colonnes : title, description, starts_at, duration_minutes (défaut 60), capacity (défaut 20), coach_id. Relations : coach (N-1), exercices et participations (1-N, cascade="all, delete-orphan"). Deux propriétés Python calculées (non stockées) : coach_name et registered_count (len(self.participations)), consommées directement par le schéma SessionRead via from_attributes=True. *Notable* : bon exemple de dénormalisation en lecture sans duplication en base ; accès implicite à self.coach/self.participations, à charger en amont (joinedload, selectinload) pour éviter le problème classique des requêtes N+1 (ce que fait effectivement le routeur, cf. §7.1.4).

PY backend/app/models/session.py (lignes 17-43)

```
class Session(Base):
    """Séance de sport (titre, horaire, durée, capacité) organisée par un coach,
    regroupant des exercices et des participations.
    """

    __tablename__ = "sport_sessions"
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(Unicode(150))
    description: Mapped[str | None] = mapped_column(UnicodeText, nullable=True)
    starts_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
    duration_minutes: Mapped[int] = mapped_column(default=60)
    capacity: Mapped[int] = mapped_column(default=20)
    coach_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    coach: Mapped["User"] = relationship(back_populates="sessions")
    exercices: Mapped[list["Exercise"]] = relationship(back_populates="session",
        cascade="all, delete-orphan")
    participations: Mapped[list["Participation"]] =
        relationship(back_populates="session", cascade="all, delete-orphan")
    @property
    def coach_name(self) -> str:
        """Nom complet du coach qui anime la séance, exposé aux schémas de
        lecture."""
        return self.coach.full_name

    @property
    def registered_count(self) -> int:
        """Nombre d'inscriptions (toutes statuts confondus) à la séance, pour
        afficher les places restantes."""
        return len(self.participations)
```

models/participation.py — Table participations (table d'association enrichie entre User et Session). Colonnes : user_id, session_id, status (Enum SQL "inscrit"|"present"|"absent"). *Notable* : aucune contrainte d'unicité (user_id, session_id) en base — rien n'empêche techniquement une double inscription si la vérification applicative (faite dans le routeur) était contournée.

PY backend/app/models/participation.py (lignes 15-27)

```

class Participation(Base):
    """Lien entre un utilisateur et une séance, avec un statut de présence
    (inscrit / présent / absent).
    """

    __tablename__ = "participations"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    session_id: Mapped[int] = mapped_column(ForeignKey("sport_sessions.id"))
    status: Mapped[str] = mapped_column(Enum("inscrit", "present", "absent",
        name="participation_status"), default="inscrit")
    user: Mapped["User"] = relationship(back_populates="participations")
    session: Mapped["Session"] = relationship(back_populates="participations")

```

models/performance.py — Table performances. Colonnes : user_id, session_id, score (Float), notes, recorded_at (défaut datetime.utcnow). *Notable* : relations user/session déclarées sans back_populates (navigation à sens unique), contrairement à Session/Participation — légère incohérence de style entre modèles.

PY backend/app/models/performance.py (lignes 16-27)

```

class Performance(Base):
    """Score enregistré pour un utilisateur sur une séance donnée, avec notes
    optionnelles."""

    __tablename__ = "performances"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    session_id: Mapped[int] = mapped_column(ForeignKey("sport_sessions.id"))
    score: Mapped[float] = mapped_column(Float)
    notes: Mapped[str | None] = mapped_column(UnicodeText, nullable=True)
    recorded_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
        default=datetime.utcnow)
    user: Mapped["User"] = relationship()
    session: Mapped["Session"] = relationship()

```

models/exercise.py — Table exercices. Colonnes : session_id, name, sets/repetitions (nullable), rest_seconds (défaut 90). *Notable* : sets est nullable en base alors que le schéma de création (ExerciseCreate) le rend obligatoire — écart de contrainte entre couches (cf. §5.3).

PY backend/app/models/exercise.py (lignes 14-25)

```

class Exercise(Base):
    """Exercice (nom, séries, répétitions, temps de repos) appartenant à une
        séance."""

    __tablename__ = "exercises"
    id: Mapped[int] = mapped_column(primary_key=True)
    session_id: Mapped[int] = mapped_column(ForeignKey("sport_sessions.id"))
    name: Mapped[str] = mapped_column(Unicode(150))
    description: Mapped[str | None] = mapped_column(UnicodeText, nullable=True)
    sets: Mapped[int | None] = mapped_column(Integer, nullable=True)
    repetitions: Mapped[int | None] = mapped_column(Integer, nullable=True)
    rest_seconds: Mapped[int] = mapped_column(Integer, default=90)
    session: Mapped["Session"] = relationship(back_populates="exercises")

```

models/goal.py — Table goals. Colonnes : user_id (FK ondelete="CASCADE"), title, metric, target_value, current_value (défaut 0), unit, due_date, created_at. Aucune relationship() déclarée (FK brute uniquement).

PY backend/app/models/goal.py (lignes 12-27)

```

class Goal(Base):
    """Objectif d'un utilisateur : métrique suivie, valeur cible/actuelle et
        échéance."""

    __tablename__ = "goals"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id",
        ondelete="CASCADE"), index=True)
    title: Mapped[str] = mapped_column(Unicode(120))
    metric: Mapped[str] = mapped_column(Unicode(40))
    target_value: Mapped[float] = mapped_column()
    current_value: Mapped[float] = mapped_column(default=0)
    unit: Mapped[str] = mapped_column(Unicode(20))
    due_date: Mapped[date | None] = mapped_column(Date, nullable=True)
    notes: Mapped[str | None] = mapped_column(UnicodeText, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

```

models/record.py — Table personal_records, classe PersonalRecord. Colonnes : user_id (cascade DB), exercise_name (chaîne libre), value, unit, achieved_at. *Notable* : exercise_name n'est **pas** une clé étrangère vers exercises — un record est associé à un nom d'exercice en texte libre, pas à une ligne réelle du catalogue, ce qui autorise incohérences et doublons de nommage.

PY backend/app/models/record.py (lignes 11-22)

```

class PersonalRecord(Base):
    """Record personnel : meilleure valeur atteinte par un utilisateur pour un
        exercice, à une date donnée."""

    __tablename__ = "personal_records"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id",
        ondelete="CASCADE"), index=True)
    exercise_name: Mapped[str] = mapped_column(Unicode(120))
    value: Mapped[float] = mapped_column()
    unit: Mapped[str] = mapped_column(Unicode(20))
    achieved_at: Mapped[datetime] = mapped_column(DateTime,
        default=datetime.utcnow)
    notes: Mapped[str | None] = mapped_column(Unicode(255), nullable=True)

```

models/program.py — Table workout_programs, classe WorkoutProgram. Colonnes : user_id (cascade DB), name, weeks (défaut 4), sessions (colonne JSON, structure libre non normalisée). *Notable* : choix « document » dans un schéma par ailleurs relationnel, pratique pour une structure encore peu stabilisée, mais sans validation stricte de son contenu interne au niveau base.

PY backend/app/models/program.py (lignes 11-24)

```

class WorkoutProgram(Base):
    """Programme d'entraînement d'un utilisateur : durée en semaines et plan de
        séances
        stocké en JSON (structure libre définie côté schémas/frontend).
    """

    __tablename__ = "workout_programs"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id",
        ondelete="CASCADE"), index=True)
    name: Mapped[str] = mapped_column(Unicode(120))
    description: Mapped[str | None] = mapped_column(UnicodeText, nullable=True)
    weeks: Mapped[int] = mapped_column(default=4)
    sessions: Mapped[list] = mapped_column(JSON, default=list)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

```

models/journal.py — Table training_journals, classe TrainingJournal. UniqueConstraint("user_id", "session_id") — seul modèle métier avec une contrainte d'unicité composite explicite en base. Colonnes : fatigue (défaut 5), mood (défaut "bien"), pain, coach_comment, updated_at (onupdate=datetime.utcnow, horodatage automatique de mise à jour).

PY backend/app/models/journal.py (lignes 12-29)

```

class TrainingJournal(Base):
    """Entrée de journal liée à un utilisateur et une séance (fatigue, humeur,
        douleurs, notes).
    Une seule entrée est autorisée par couple (utilisateur, séance).
    """

    __tablename__ = "training_journals"
    __table_args__ = (UniqueConstraint("user_id", "session_id",
        name="UQ_Journal_User_Session"),)

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id",
        ondelete="CASCADE"), index=True)
    session_id: Mapped[int] = mapped_column(ForeignKey("sport_sessions.id",
        ondelete="CASCADE"), index=True)
    notes: Mapped[str | None] = mapped_column(UnicodeText, nullable=True)
    fatigue: Mapped[int] = mapped_column(Integer, default=5)
    mood: Mapped[str] = mapped_column(Unicode(30), default="bien")
    pain: Mapped[str | None] = mapped_column(Unicode(255), nullable=True)
    coach_comment: Mapped[str | None] = mapped_column(UnicodeText, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime,
        default=datetime.utcnow, onupdate=datetime.utcnow)

```

models/notification.py — Table notifications. Colonnes : title, message, kind (chaîne libre, non contrainte par un Enum — incohérence avec role/status qui eux le sont), is_read (défaut False).

PY backend/app/models/notification.py (lignes 11-22)

```

class Notification(Base):
    """Notification destinée à un utilisateur (titre, message, type, statut lu/non
        lu)."""

    __tablename__ = "notifications"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id",
        ondelete="CASCADE"), index=True)
    title: Mapped[str] = mapped_column(Unicode(120))
    message: Mapped[str] = mapped_column(UnicodeText)
    kind: Mapped[str] = mapped_column(String(20), default="info")
    is_read: Mapped[bool] = mapped_column(Boolean, default=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

```

models/refresh_token.py — Table refresh_tokens. Colonnes : token_hash (unique, indexé — jamais le jeton en clair, cf. §6.1), expires_at, revoked_at (nullable). Propriété calculée is_valid (revoked_at is None et expires_at dans le futur). *Notable* : relation user déclarée par chaîne (relationship("User")) plutôt qu'avec l'annotation Mapped[...] utilisée ailleurs — incohérence de style mineure.

PY backend/app/models/refresh_token.py (lignes 12-32)

```

class RefreshToken(Base):
    """Jeton de rafraîchissement (stocké sous forme de hachage, jamais en clair)
    associé
    à un utilisateur, avec sa date d'expiration et une éventuelle date de
    révocation.
    """

    __tablename__ = "refresh_tokens"

    id: Mapped[int] = mapped_column(primary_key=True)
    token_hash: Mapped[str] = mapped_column(String(64), unique=True, index=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id",
        ondelete="CASCADE"))
    expires_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
    revoked_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True),
        nullable=True)
    user = relationship("User")
    @property
    def is_valid(self) -> bool:
        """Indique si le jeton est encore utilisable : non révoqué et non
        expiré."""
        expires_at = self.expires_at if self.expires_at.tzinfo else
            self.expires_at.replace(tzinfo=timezone.utc)
        return self.revoked_at is None and expires_at > datetime.now(timezone.utc)

```

Synthèse : deux stratégies de suppression en cascade coexistent dans le projet — voir §5.2 pour le détail et la discussion.

7.1.3 Schémas Pydantic (app/schemas/)

schemas/__init__.py — Ré-exporte uniquement les schémas de auth, user, session, participation, performance (5 domaines sur 10) ; les autres (exercise, goals, program, journal, notification, statistics) s'importent directement depuis leur module — couverture partielle et incohérente du fichier barrel, comparé à models/__init__.py qui, lui, réexporte tout.

schemas/user.py — UserCreate : email: EmailStr, full_name (2-150 caractères), password (12-128 caractères), role (chaîne libre par défaut "sportif", non contrainte par Literal). Validateurs Pydantic : normalisation de l'email (minuscule, espaces retirés), normalisation du nom, et **rejet des mots de passe faibles** (reject_weak_passwords : liste noire de mots de passe communs, interdit les espaces, exige minuscule + majuscule + chiffre). UserUpdate : tous les champs optionnels (la restriction « seul un admin peut changer role/is_active » est appliquée dans le routeur, pas dans le schéma). UserRead (from_attributes=True) : n'expose jamais hashed_password — exemple concret du schéma de sortie comme barrière contre la fuite de données sensibles (§2.5). *Notable* : role n'étant pas contraint par un Literal côté Pydantic, une valeur invalide échouerait seulement à l'insertion SQL (erreur peu explicite) plutôt qu'avec une 422 lisible — écart de contrainte déjà signalé au §5.3.

PY backend/app/schemas/user.py (lignes 30-46)

```

@field_validator("password")
@classmethod
def reject_weak_passwords(cls, value: str) -> str:
    """Rejette les mots de passe trop courants ou ne respectant pas la
    politique
    de complexité (minuscule, majuscule, chiffre, pas d'espace).
    """
    if value.lower() in {"password123456", "azerty123456", "qwerty123456"}:
        raise ValueError("Mot de passe trop courant")
    if any(character.isspace() for character in value):
        raise ValueError("Le mot de passe ne doit pas contenir d'espaces")
    if not any(character.islower() for character in value):
        raise ValueError("Le mot de passe doit contenir une minuscule")
    if not any(character.isupper() for character in value):
        raise ValueError("Le mot de passe doit contenir une majuscule")
    if not any(character.isdigit() for character in value):
        raise ValueError("Le mot de passe doit contenir un chiffre")
    return value

```

schemas/session.py — SessionBase (champs communs, duration_minutes et capacity > 0) ; SessionCreate ajoute coach_id ; SessionUpdate (tous champs optionnels) ; SessionRead ajoute id, coach_id, coach_name, registered_count — ces deux derniers directement mappés depuis les propriétés Python calculées du modèle (cf. models/session.py).

schemas/participation.py — ParticipationCreate (user_id, session_id) ; ParticipationUpdate (status: str, non contraint par Literal) ; ParticipationRead.

PY backend/app/schemas/participation.py (lignes 6-16)

```

class ParticipationCreate(BaseModel):
    """Données requises pour inscrire un utilisateur à une séance."""

    user_id: int
    session_id: int

class ParticipationUpdate(BaseModel):
    """Mise à jour du statut de présence d'une participation."""

    status: str

```

schemas/performance.py — PerformanceCreate (score: float = Field(ge=0)) ; PerformanceRead ajoute id, recorded_at ; PerformanceUpdate (champs optionnels). *Notable* : la non-négativité du score n'est imposée qu'à ce niveau (aucune contrainte CHECK équivalente côté colonne SQL).

schemas/exercise.py — ExerciseCreate : sets **obligatoire** (Field(gt=0, le=30)) — incohérence déjà relevée avec la colonne nullable côté modèle. repetitions optionnel, rest_seconds borné (0-3600s).

PY backend/app/schemas/exercise.py (lignes 6-13)

```
class ExerciseCreate(BaseModel):
    """Données requises pour créer un exercice."""

    name: str = Field(min_length=1, max_length=150)
    description: str | None = Field(default=None, max_length=500)
    sets: int = Field(gt=0, le=30)
    repetitions: int | None = Field(default=None, gt=0, le=500)
    rest_seconds: int = Field(default=90, ge=0, le=3600)
```

schemas/goals.py — Regroupe GoalCreate/GoalRead (target_value: float = Field(gt=0), empêchant un objectif à valeur cible nulle ou négative) et RecordCreate/RecordRead. *Notable* : organisation asymétrique par rapport à models/ (deux entités distinctes, goal.py/record.py, regroupées ici dans un seul fichier goals.py).

PY backend/app/schemas/goals.py (lignes 8-17)

```
class GoalCreate(BaseModel):
    """Données requises pour créer ou mettre à jour un objectif."""

    title: str = Field(min_length=2, max_length=120)
    metric: str = Field(min_length=2, max_length=40)
    target_value: float = Field(gt=0)
    current_value: float = Field(default=0, ge=0)
    unit: str = Field(min_length=1, max_length=20)
    due_date: date | None = None
    notes: str | None = Field(default=None, max_length=500)
```

schemas/program.py — ProgramCreate : weeks borné 1-52 (testé par test_program_limits_week_count), sessions: list[dict] — structure interne non validée finement (documenté en commentaire comme « définie côté frontend »).

schemas/journal.py — JournalCreate (fatigue: int borné 1-10, testé par test_journal_rejects_invalid_fatigue) n'expose pas coach_comment (un sportif ne peut pas le renseigner à la création) ; JournalUpdate l'inclut, la restriction de *qui* peut le modifier étant déléguée au routeur (séparation validation/autorisation, cf. §7.1.4).

PY backend/app/schemas/journal.py (lignes 8-27)

```

class JournalCreate(BaseModel):
    """Données requises pour créer une entrée de journal (par le sportif ou le
        coach)."""

    session_id: int
    notes: str | None = Field(default=None, max_length=2000)
    fatigue: int = Field(default=5, ge=1, le=10)
    mood: str = Field(default="bien", min_length=2, max_length=30)
    pain: str | None = Field(default=None, max_length=255)

class JournalUpdate(BaseModel):
    """Champs modifiables d'une entrée de journal. Tous optionnels : seuls les
        champs
        fournis sont mis à jour (voir la logique de restriction par rôle dans le
        routeur).
    """
    notes: str | None = Field(default=None, max_length=2000)
    fatigue: int | None = Field(default=None, ge=1, le=10)
    mood: str | None = Field(default=None, min_length=2, max_length=30)
    pain: str | None = Field(default=None, max_length=255)
    coach_comment: str | None = Field(default=None, max_length=2000)

```

schemas/notification.py — NotificationCreate/NotificationRead, kind en chaîne libre non documentée par un jeu de valeurs fermé.

schemas/statistics.py — StatisticsRead : schéma de sortie pur (sans from_attributes, car calculé par agrégation plutôt que mappé depuis une seule table) — total_sessions, upcoming_sessions, total_participations, attended_sessions, total_performances, average_score (optionnel, None si aucune performance encore enregistrée).

schemas/auth.py — Token (paire access/refresh + token_type), TokenRefresh, LogoutRequest (structure identique à TokenRefresh — un seul champ refresh_token, duplication de schéma évitable), PasswordChange (12-128 caractères, mais ne réapplique pas les règles de complexité de reject_weak_passwords — le changement de mot de passe est donc, en l'état, moins strict que la création de compte sur ce point précis).

PY backend/app/schemas/auth.py (lignes 1-fin)

```

"""Schémas Pydantic pour l'authentification : jetons JWT et changement de mot de
passe."""

from pydantic import BaseModel
from pydantic import Field

class Token(BaseModel):
    """Paire de jetons JWT retournée après une connexion ou un rafraîchissement
    réussi."""

    access_token: str
    refresh_token: str
    token_type: str = "bearer"

class TokenRefresh(BaseModel):
    """Requête d'échange d'un refresh token contre une nouvelle paire de jetons."""
    refresh_token: str

class LogoutRequest(BaseModel):
    """Requête de déconnexion : refresh token à révoquer."""

    refresh_token: str

class PasswordChange(BaseModel):
    """Requête de changement de mot de passe (mot de passe actuel + nouveau)."""

    current_password: str = Field(min_length=12, max_length=128)
    new_password: str = Field(min_length=12, max_length=128)

```

7.1.4 Routeurs FastAPI (app/routers/)

routers/auth.py — Endpoints : POST /auth/register (201, limité en débit, rejette role hors {coach, sportif} — impossible de créer un compte admin par ce chemin ; 409 si email déjà pris) ; POST /auth/login (limité en débit, OAuth2PasswordRequestForm, 401 générique en cas d'échec, cf. §6.1) ; POST /auth/refresh (rotation du jeton, 401 si invalide) ; POST /auth/logout (204, idempotent) ; POST /auth/change-password (400 si mot de passe actuel incorrect ou nouveau identique à l'ancien). *Notable* : journalisation (logging) des échecs et succès de connexion, utile pour l'audit ; **défaut de code mort relevé** — dans refresh(), un raise HTTPException(401, ...) inconditionnel suit un bloc if/elif qui a déjà couvert tous les cas possibles, rendant cette dernière ligne inatteignable (sans conséquence fonctionnelle, mais à nettoyer). Le seul moyen observé de créer un compte admin est la promotion d'un compte existant via PATCH /users/{id} par un autre admin.

PY backend/app/routers/auth.py (lignes 81-103)

```

@router.post("/refresh", response_model=Token)
def refresh(data: TokenRefresh, db: Session = Depends(get_db)):
    """Échange un refresh token valide contre une nouvelle paire de jetons (POST
    /auth/refresh).
    Applique une rotation : l'ancien refresh token est révoqué et un nouveau est
    émis.
    """
    try:
        payload = decode_token(data.refresh_token)
        if payload.get("type") != "refresh":
            raise ValueError
        user = db.get(User, int(payload["sub"]))
        stored = db.scalar(select(RefreshToken).where(RefreshToken.token_hash ==
            _token_hash(data.refresh_token)))
        if not stored or not stored.is_valid or not user or stored.user_id !=
            user.id:
            user = None
        elif user.is_active:
            stored.revoked_at = datetime.now(timezone.utc)
            new_refresh = _issue_refresh_token(user.id, db)
            db.commit()
            return {"access_token": create_access_token(str(user.id)),
                "refresh_token": new_refresh}
    except (JWTError, ValueError, TypeError, KeyError):
        user = None
    if not user or not user.is_active:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Refresh token invalide")
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Refresh token invalide")

```

(le dernier raise est le code mort signalé dans le tableau : la ligne qui le précède couvre déjà tous les cas où l'exécution peut l'atteindre.)

routers/users.py — GET /users/me (profil courant) ; GET /users/ et GET /users/athletes (réservés coach/admin) ; PATCH /users/{id} (chacun peut modifier son propre profil, seul un admin peut modifier un autre compte, changer un rôle, ou (dés)activer un compte ; 400 si le nouveau rôle n'est pas valide). *Notable* : commentaire de portabilité SQL Server explicite dans le code (`User.is_active == True` plutôt que `.is_(True)`), ce dernier générant un SQL que SQL Server rejette) — illustration concrète d'un piège d'portabilité entre moteurs de bases de données que l'ORM ne masque pas toujours totalement.

PY backend/app/routers/users.py (lignes 27-31)

```

@router.get("/athletes", response_model=list[UserRead],
    dependencies=[Depends(require_roles("coach", "admin"))])
def list_athletes(db: Session = Depends(get_db)):
    """Liste les sports actifs (GET /users/athletes), triés par nom. Réservé aux
    comptes coach et admin."""
    # SQL Server n'accepte pas "IS 1" (produit par .is_(True)) : il faut comparer
    avec "=".
    return db.scalars(select(User).where(User.role == "sportif", User.is_active ==
        True).order_by(User.full_name)).all() # noqa: E712

```

routers/sessions.py — GET /sessions/ et GET /sessions/{id} **publics** (aucune authentification requise, seules routes de ce type dans le projet) ; POST/PATCH/DELETE

réservés coach (propriétaire) ou admin, avec double garde temporelle : 400 si la nouvelle date est déjà passée, 409 si la séance existante est déjà passée (modification/ suppression alors interdite). Utilise `joinedload/selectinload` pour charger coach et participations en une seule requête (évite le problème des requêtes N+1, §2.4). Notifie automatiquement tous les inscrits (création de Notification) en cas de modification d'horaire ou d'annulation.

PY `backend/app/routers/sessions.py` (lignes 69-96)

```
@router.patch("/{session_id}", response_model=SessionRead)
def update_session(session_id: int, data: SessionUpdate, db: Session =
    Depends(get_db), user: User = Depends(require_roles("coach", "admin"))):
    """Met à jour une séance (PATCH /sessions/{session_id}). Réserve au coach
        responsable
        ou à un admin. Impossible de modifier une séance déjà passée, ni de la
        reprogrammer dans le passé.
    """
    item = db.get(SportSession, session_id)
    if not item:
        raise HTTPException(404, "Séance introuvable")
    if user.role == "coach" and item.coach_id != user.id:
        raise HTTPException(403, "Vous ne gérez pas cette séance")
    if is_past(item.starts_at):
        raise HTTPException(409, "Une séance passée ne peut plus être modifiée")
    if data.starts_at and is_past(data.starts_at):
        raise HTTPException(400, "Une séance doit être planifiée dans le futur")
    changes = data.model_dump(exclude_unset=True)
    schedule_changed = "starts_at" in changes and changes["starts_at"] !=
        item.starts_at
    for key, value in changes.items():
        setattr(item, key, value)
    if changes:
        message = (
            f"Nouvel horaire : {item.starts_at:%d/%m/%Y à %H:%M}."
            if schedule_changed
            else "Les informations de la séance ont été mises à jour."
        )
        _notify_participants(db, item, f"Séance modifiée : {item.title}", message,
            kind="warning" if schedule_changed else "info")
    db.commit()
    db.refresh(item)
    return item
```

`routers/exercises.py` — Routeur imbriqué sous `/sessions/{session_id}/exercises` (bon exemple REST de ressource enfant). GET public ; POST/DELETE réservés au coach responsable de la séance ou à un admin. Pas de PATCH (un exercice existant ne se modifie pas, seulement se supprime et se recrée).

PY `backend/app/routers/exercises.py` (lignes 23-42)

```

@router.post("/", response_model=ExerciseRead, status_code=201)
def create_exercise(
    session_id: int,
    data: ExerciseCreate,
    db: Session = Depends(get_db),
    user: User = Depends(require_roles("coach", "admin")),
):
    """Ajoute un exercice à une séance (POST /sessions/{session_id}/exercices/).
    Réservé aux comptes coach (propriétaire de la séance) et admin.
    """
    session = db.get(SportSession, session_id)
    if not session:
        raise HTTPException(404, "Séance introuvable")
    if user.role == "coach" and session.coach_id != user.id:
        raise HTTPException(403, "Vous ne gérez pas cette séance")
    exercise = Exercise(session_id=session_id, **data.model_dump())
    db.add(exercise)
    db.commit()
    db.refresh(exercise)
    return exercise

```

routers/participations.py — GET filtré par rôle (admin = tout, coach = participations des séances qu'il encadre via jointure, sportif = les siennes — pattern répété dans plusieurs routeurs, cf. §6.2). POST : 409 si séance passée, complète, ou double inscription. PATCH : seul le coach de la séance ou un admin peut passer un statut à *present/absent* ; un sportif ne peut que se remettre à *inscrit*. DELETE : 409 si la séance a déjà commencé. *Notable* : la vérification de capacité (comptage puis insertion) n'est pas protégée par une transaction verrouillante — deux inscriptions quasi simultanées proches de la limite pourraient théoriquement dépasser la capacité affichée (condition de concurrence à corriger pour une mise en production réelle).

PY backend/app/routers/participations.py (lignes 33-57)

```

@router.post("/", response_model=ParticipationRead, status_code=201)
def create_participation(data: ParticipationCreate, db: Session = Depends(get_db),
    user: User = Depends(get_current_user)):
    """Inscrit un utilisateur à une séance (POST /participations/). Un sportif ne
    peut
    s'inscrire que lui-même. Rejette les séances passées, complètes, ou une double
    inscription.
    """
    if user.role == "sportif" and data.user_id != user.id:
        raise HTTPException(403, "Vous ne pouvez inscrire qu'un compte")
    if not db.get(User, data.user_id) or not db.get(SportSession, data.session_id):
        raise HTTPException(404, "Utilisateur ou séance introuvable")
    session = db.get(SportSession, data.session_id)
    if is_past(session.starts_at):
        raise HTTPException(409, "Impossible de s'inscrire à une séance passée")
    registrations = db.scalar(
        select(func.count(Participation.id)).where(Participation.session_id ==
            data.session_id)

```

```

    ) or 0
    if registrations >= session.capacity:
        raise HTTPException(409, "Cette séance est complète")
    if db.scalar(select(Participation).where(Participation.user_id ==
        data.user_id, Participation.session_id == data.session_id)):
        raise HTTPException(409, "Participation déjà existante")
    item = Participation(**data.model_dump())
    db.add(item)
    db.commit()
    db.refresh(item)
    return item

```

(le comptage registrations puis l'insertion ne sont pas dans une même transaction verrouillée — c'est la condition de concurrence théorique signalée au chapitre 10.)

`routers/performances.py` — Mêmes règles de portée par rôle. POST exige que le sportif visé soit inscrit (Participation existante) à la séance avant d'accepter une performance — garantit la cohérence « pas de performance sans participation ».

PY `backend/app/routers/performances.py` (lignes 33-56)

```

@router.post("/", response_model=PerformanceRead, status_code=201)
def create_performance(data: PerformanceCreate, db: Session = Depends(get_db),
    user: User = Depends(get_current_user)):
    """Enregistre une performance pour un sportif sur une séance (POST
        /performances/).
    Un sportif ne peut enregistrer que ses propres performances ; le sportif visé
    doit
    être inscrit (participant) à la séance.
    """
    if user.role == "sportif" and data.user_id != user.id:
        raise HTTPException(403, "Vous ne pouvez enregistrer que vos propres
            performances")
    session = db.get(SportSession, data.session_id)
    if not session:
        raise HTTPException(404, "Séance introuvable")
    if user.role == "coach" and session.coach_id != user.id:
        raise HTTPException(403, "Vous ne gérez pas cette séance")
    participation = db.scalar(select(Participation).where(
        Participation.session_id == data.session_id,
        Participation.user_id == data.user_id,
    ))
    if not participation:
        raise HTTPException(409, "Le sportif doit être inscrit à la séance")
    item = Performance(**data.model_dump())
    db.add(item)
    db.commit()
    db.refresh(item)
    return item

```

`routers/statistics.py` — Un seul endpoint, GET `/statistics/me`, dont le calcul se branche selon `user.role` (coach/admin : agrégats sur les séances qu'ils encadrent, ou toutes pour un admin ; sportif : agrégats sur ses propres données). **Incohérence relevée** : `average_score` est calculé sur *l'ensemble* des performances de tous les utilisateurs pour un coach/admin (`Performance.user_id == user.id if user.role == "sportif"`

else True), alors que les autres métriques de la même réponse (total_sessions, total_participations...) sont bien restreintes aux séances gérées par ce coach — un score moyen non représentatif pourrait être affiché à un coach.

PY backend/app/routers/statistics.py (lignes 21-61)

```
@router.get("/me", response_model=StatisticsRead)
def my_statistics(db: Session = Depends(get_db), user: User =
    Depends(get_current_user)):
    """Calcule les statistiques de l'utilisateur connecté (GET /statistics/me).
    Pour un coach/admin : statistiques sur les séances qu'il encadre (toutes
        séances pour un admin).
    Pour un sportif : statistiques sur ses propres participations et performances.
    """
    now = datetime.now(timezone.utc)
    if user.role in {"coach", "admin"}:
        # `True` comme filtre neutralise le WHERE pour un admin (accès à toutes
            les séances).
        session_filter = True if user.role == "admin" else SportSession.coach_id
            == user.id
        total_sessions =
            db.scalar(select(func.count(SportSession.id)).where(session_filter))
            or 0
        upcoming_sessions =
            db.scalar(select(func.count(SportSession.id)).where(session_filter,
                SportSession.starts_at > now)) or 0
        participation_filter = True if user.role == "admin" else
            SportSession.coach_id == user.id
        participation_query =
            select(Participation).join(SportSession).where(participation_filter)
        performance_query =
            select(Performance).join(SportSession).where(participation_filter)
    else:
        total_sessions =
            db.scalar(select(func.count(Participation.id)).where(Participation.user
                _id == user.id)) or 0
        upcoming_sessions = db.scalar(
            select(func.count(Participation.id)).join(SportSession).where(
                Participation.user_id == user.id, SportSession.starts_at > now
            )
        ) or 0
        participation_query = select(Participation).where(Participation.user_id ==
```

```

        user.id)
    performance_query = select(Performance).where(Performance.user_id ==
        user.id)

    total_participations =
        db.scalar(select(func.count()).select_from(participation_query.subquery()))
        or 0
    attended_sessions = db.scalar(
        select(func.count()).select_from(participation_query.where(Participation.st
            atus == "present").subquery())
    ) or 0
    total_performances =
        db.scalar(select(func.count()).select_from(performance_query.subquery()))
        or 0
    average_score = db.scalar(select(func.avg(Performance.score)).where(
        Performance.user_id == user.id if user.role == "sportif" else True
    ))
    return StatisticsRead(
        total_sessions=total_sessions,
        upcoming_sessions=upcoming_sessions,
        total_participations=total_participations,
        attended_sessions=attended_sessions,
        total_performances=total_performances,
        average_score=round(float(average_score), 2) if average_score is not None
        else None,
    )

```

(ligne `average_score = ...` : c'est ici que le filtre `else True` rend le score moyen non borné aux séances du coach, contrairement à toutes les autres métriques de la même fonction — cf. §10.1.)

`routers/goals.py` — CRUD des objectifs (GET/POST/PATCH/DELETE `/goals`) et des records personnels (GET/POST `/records`), tous deux implicitement filtrés par propriété (`user_id == user.id`), sans `require_roles` (ressource strictement personnelle). *Notable* : aucun DELETE `/records/{id}` (asymétrie avec les objectifs) ; `update_goal` applique `data.model_dump()` sans `exclude_unset=True`, contrairement aux autres routeurs de mise à jour partielle — un PATCH incomplet côté client pourrait réécraser des champs non transmis avec leur valeur par défaut du schéma.

PY `backend/app/routers/goals.py` (lignes 34-44)

```

@router.patch("/goals/{goal_id}", response_model=GoalRead)
def update_goal(goal_id: int, data: GoalCreate, db: Session = Depends(get_db),
    user: User = Depends(get_current_user)):
    """Met à jour un objectif appartenant à l'utilisateur connecté (PATCH
        /goals/{goal_id})."""
    item = db.get(Goal, goal_id)
    if not item or item.user_id != user.id:
        raise HTTPException(404, "Objectif introuvable")
    for key, value in data.model_dump().items():
        setattr(item, key, value)
    db.commit()
    db.refresh(item)
    return item

```

(`data.model_dump()` sans `exclude_unset=True`, contrairement aux autres PATCH du projet — cf. §10.3.)

`routers/programs.py` — GET/POST/DELETE `/programs`, filtré par propriété. Pas de PATCH (un programme ne se modifie pas après création dans cette version).

`routers/journal.py` — Logique d'autorisation la plus fine du projet : un sportif ne peut jamais modifier `coach_comment` (retiré des champs modifiables) ; un coach, à l'inverse, ne peut modifier **que** `coach_comment` (tout autre champ envoyé est silencieusement ignoré) — séparation des droits *par champ*, pas seulement par ressource. 409 si une entrée existe déjà pour le couple (utilisateur, séance) — même vérification applicative « SELECT puis INSERT », donc même risque théorique de concurrence que pour les participations.

PY `backend/app/routers/journal.py` (lignes 63-89)

```
@router.patch("/{journal_id}", response_model=JournalRead)
def update_journal(journal_id: int, data: JournalUpdate, db: Session =
    Depends(get_db), user: User = Depends(get_current_user)):
    """Met à jour une entrée de journal (PATCH /journal/{journal_id}).
    Un sportif ne peut modifier que son propre journal et ne peut pas toucher au
    commentaire du coach ; un coach responsable de la séance ne peut modifier que
    ce
    commentaire.
    """
    item = db.get(TrainingJournal, journal_id)
    if not item:
        raise HTTPException(404, "Entrée de journal introuvable")
    session = db.get(SportSession, item.session_id)
    if user.role == "sportif" and item.user_id != user.id:
        raise HTTPException(403, "Vous ne pouvez modifier que votre journal")
    if user.role == "coach" and (not session or session.coach_id != user.id):
        raise HTTPException(403, "Vous ne gérez pas cette séance")
    changes = data.model_dump(exclude_unset=True)
    if user.role == "sportif":
        # Le champ coach_comment n'est modifiable que par le coach.
        changes.pop("coach_comment", None)
    if user.role == "coach":
        # Le coach ne peut modifier que le commentaire, jamais le reste de
        # l'entrée.
        changes = {"coach_comment": changes["coach_comment"]} if "coach_comment"
            in changes else {}
    for key, value in changes.items():
        setattr(item, key, value)
    db.commit()
    db.refresh(item)
    return item
```

`routers/notifications.py` — GET `/notifications` (les siennes) ; PATCH `/notifications/{id}/read`. POST `/notifications` (réservé admin) comporte un **défaut fonctionnel reconnu dans le code source lui-même** (commentaire de la fonction) : la notification créée cible toujours l'administrateur appelant (`user_id=user.id`) plutôt qu'un destinataire éventuellement fourni dans le corps de la requête — contrairement aux notifications automatiques de `sessions.py`, qui, elles, ciblent correctement chaque participant.

PY backend/app/routers/notifications.py (lignes 22-33)

```
@router.post("", response_model=NotificationRead, status_code=201)
def create_notification(data: NotificationCreate, db: Session = Depends(get_db),
    user: User = Depends(require_roles("admin"))):
    """Crée une notification (POST /notifications). Réservé aux comptes admin.

    Note : la notification est créée avec l'id de l'admin appelant comme `user_id`,
    et non celui d'un destinataire fourni dans `data` — comportement actuel du
    code.
    """
    item = Notification(user_id=user.id, **data.model_dump())
    db.add(item)
    db.commit()
    db.refresh(item)
    return item
```

(le commentaire de la docstring documente lui-même le défaut relevé au chapitre 10 : `user_id=user.id` cible toujours l'admin appelant.)

Synthèse transversale du backend : le filtrage par rôle (admin/coach/sportif) est un pattern répété quasi à l'identique dans quatre routeurs plutôt que factorisé ; deux défauts fonctionnels confirmés existent (`average_score` non filtré, destinataire ignoré dans la création manuelle de notification) ; l'absence de verrou explicite sur les vérifications « existe déjà »/« capacité atteinte » constitue un risque théorique de concurrence à corriger avant une mise en production réelle à fort trafic. Ces observations n'enlèvent rien à la cohérence d'ensemble de la couche sécurité (Argon2, JWT avec rotation, RBAC, validation Pydantic systématique), qui reste solide.

7.2 Frontend (frontend/src/app/)

7.2.1 Bootstrap et coquille applicative (core/)

core/api.config.ts — Exporte `API_URL = 'http://localhost:8000'`, constante unique utilisée par tous les services HTTP. *Notable* : valeur codée en dur, sans fichier `environment.ts/environment.prod.ts` — un déploiement hors `localhost:8000` demande de modifier ce fichier.

TS frontend/src/app/core/api.config.ts (lignes 1-fin)

```
/** Base URL of the local FastAPI server (see backend/SETUP_LOCAL.md). */
export const API_URL = 'http://localhost:8000';
```

core/app.config.ts — Bootstrap de l'application standalone : `provideZoneChangeDetection({ eventCoalescing: true })`, `provideRouter(routes)`, `provideHttpClient(withInterceptors([authInterceptor]))`, `provideAnimationsAsync()`. *Notable* : un commentaire du code explique un point d'architecture propre à cette version d'Angular — depuis Angular 20+, `bootstrapApplication` démarre en mode « zoneless » par défaut même en présence de `zone.js` dans les polyfills ; sans l'appel explicite à `provideZoneChangeDetection`, une

mutation classique dans un callback asynchrone ne déclencherait plus de rafraîchissement de la vue. C'est ce même besoin qui justifie l'opérateur `markForCheck` utilisé ailleurs (voir plus bas).

TS frontend/src/app/core/app.config.ts (lignes 13-27)

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { provideRouter } from '@angular/router';
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
import { routes } from './app.routes';
import { authInterceptor } from '@features/auth/auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])),
    provideAnimationsAsync()
  ]
};
```

`core/app.routes.ts` — Table de routage (16 entrées) associant chaque URL à son composant *standalone* et à ses gardes (`authGuard` pour tout écran authentifié, `coachGuard` pour `/workouts/new`, `/athletes`, `/athletes/:id`, `adminGuard` pour `/admin/users`). Tous les composants sont importés statiquement (pas de `loadComponent`, donc pas de découpage du bundle par route). La route `**` redirige vers `/dashboard` plutôt que vers une page 404 dédiée.

TS frontend/src/app/core/app.routes.ts (lignes 23-41)

```
export const routes: Routes = [
  { path: 'login', component: LoginComponent },
  { path: 'dashboard', component: DashboardComponent, canActivate: [authGuard] },
  { path: 'sessions', component: SessionsComponent, canActivate: [authGuard] },
  { path: 'workouts/new', component: WorkoutCreateComponent, canActivate:
    [coachGuard] },
  { path: 'calendar', component: CalendarComponent, canActivate: [authGuard] },
  { path: 'participations', component: ParticipationsComponent, canActivate:
    [authGuard] },
  { path: 'performances', component: PerformancesComponent, canActivate:
    [authGuard] },
  { path: 'profile', component: ProfileComponent, canActivate: [authGuard] },
  { path: 'goals', component: GoalsComponent, canActivate: [authGuard] },
  { path: 'programs', component: ProgramsComponent, canActivate: [authGuard] },
  { path: 'notifications', component: NotificationsComponent, canActivate:
    [authGuard] },
  { path: 'athletes', component: AthletesComponent, canActivate: [coachGuard] },
  { path: 'athletes/:id', component: AthleteDetailComponent, canActivate:
```

```

    [coachGuard] },
    { path: 'journal', component: JournalComponent, canActivate: [authGuard] },
    { path: 'admin/users', component: AdminUsersComponent, canActivate: [adminGuard]
    },
    { path: '', pathMatch: 'full', redirectTo: 'dashboard' },
    { path: '**', redirectTo: 'dashboard' }
  ];

```

core/auth.guard.ts — Trois gardes fonctionnels (CanActivateFn, API moderne Angular plutôt que des classes implements CanActivate) : authGuard, coachGuard, adminGuard, chacun retournant un UrlTree de redirection (/login ou /dashboard) en cas de refus, pattern recommandé pour éviter une double navigation. Rappel théorique (§2.6.3, §2.8) : ces gardes ne sont qu'un confort d'expérience utilisateur, la vérification qui compte est refaite côté serveur.

TS frontend/src/app/core/auth.guard.ts (lignes 10-29)

```

export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  return auth.isAuthenticated() ? true : inject(Router).createUrlTree(['/login']);
};

/** Empêche un sportif d'ouvrir les écrans réservés au coach, même par URL. */
export const coachGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (!auth.isAuthenticated()) return router.createUrlTree(['/login']);
  return auth.isCoachOrAdmin() ? true : router.createUrlTree(['/dashboard']);
};

/** Réserve les écrans de gestion des comptes aux administrateurs. */
export const adminGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (!auth.isAuthenticated()) return router.createUrlTree(['/login']);
  return auth.isAdmin() ? true : router.createUrlTree(['/dashboard']);
};

```

core/mark-for-check.operator.ts — Opérateur RxJS « maison » : markForCheck<T>(cd: ChangeDetectorRef) relaie chaque valeur/erreur d'un flux tout en appelant cd.markForCheck(), pour forcer Angular à revérifier un composant après une mutation d'état effectuée dans un callback .subscribe() (nécessaire dans le mode de détection de changement de cette version d'Angular — cf. app.config.ts). Utilisé systématiquement dans les composants qui n'utilisent pas AsyncPipe.

TS frontend/src/app/core/mark-for-check.operator.ts (lignes 16-22)

```
export function markForCheck<T>(cd: ChangeDetectorRef): OperatorFunction<T, T> {
  return (source: Observable<T>) => new Observable<T>((observer) =>
    source.subscribe({
      next: (value) => { observer.next(value); cd.markForCheck(); },
      error: (error) => { observer.error(error); cd.markForCheck(); },
      complete: () => { observer.complete(); cd.markForCheck(); }
    }));
}
```

`core/app.component.ts` — Composant racine (app-root), qui affiche la coquille de l'application (rail de navigation, profil, bascule de thème) et héberge le `<router-outlet>`. Services injectés : `AuthService`, `ThemeService`, `UserService`, `SessionService`, `ParticipationService`, `ToastService`. `currentUser$` combine un signal (`toObservable(auth.isAuthenticated)`) avec `switchMap` (recharge `/users/me` à chaque connexion/déconnexion) et `shareReplay` (évite un double appel HTTP entre l'affichage du profil et la vérification des rappels). Méthode privée `checkUpcomingReminders(userId)` : croise participations et séances (`forkJoin`), et affiche un toast d'information pour toute séance à laquelle l'utilisateur est inscrit et qui commence dans moins de 3 heures (`REMINDER_WINDOW_MS`), une seule fois par séance grâce à une clé `localStorage` (`reminder_shown_${id}`). Le rail de navigation est structuré en sections (Planning, Suivi, Ressources, Équipe, Compte), chacune conditionnée par le rôle courant. *Notable* : mélange assumé de deux styles réactifs dans un même composant — signals pour l'état global (auth, thème) et `Observable+AsyncPipe` pour les données utilisateur ; la logique de visibilité des liens de navigation duplique celle des guards (cohérent, mais à deux endroits).

TS `frontend/src/app/core/app.component.ts` (lignes 184-204)

```
/**
 * Affiche un rappel (une seule fois par séance, via localStorage) pour chaque
 * séance à
 * laquelle le sportif est inscrit et qui commence dans les prochaines heures.
 */
private checkUpcomingReminders(userId: number): void {
  forkJoin([this.participations.list(),
    this.sessions.list()]).subscribe([participations, sessions] => {
    const now = Date.now();
    const upcoming = participations.filter((p) => p.user_id === userId &&
      p.status !== 'absent');
    for (const participation of upcoming) {
      const session = sessions.find((item) => item.id ===
        participation.session_id);
      if (!session) continue;
      const delta = new Date(session.starts_at).getTime() - now;
      if (delta <= 0 || delta > REMINDER_WINDOW_MS) continue;
      const key = `reminder_shown_${session.id}`;
```

```

        if (localStorage.getItem(key)) continue;
        localStorage.setItem(key, '1');
        const hours = Math.round(delta / (60 * 60 * 1000));
        this.toast.info(`Rappel : « ${session.title} » commence ${hours <= 1 ?
            'bientôt' : 'dans ' + hours + ' h'}.`);
    }
});
}

```

7.2.2 Authentification (features/auth/)

`auth.service.ts` — Service central de l'authentification. Champ `isAuthenticated` = `signal(Boolean(localStorage.getItem('access_token')))`. `login(email, password)` : poste au format `application/x-www-form-urlencoded` (format attendu par `OAuth2PasswordRequestForm` côté FastAPI), stocke les jetons puis enchaîne (`switchMap`) un appel à `UserService.me()` pour mémoriser le rôle (`user_role`) dans le `localStorage` — un commentaire du code souligne explicitement que le rôle vient toujours du profil renvoyé par le serveur, jamais d'une supposition côté client. `logout()` efface les jetons/rôle et navigue vers `/login`. `isCoachOrAdmin()/isAdmin()` lisent `user_role` en synchrone (pas de signal dédié pour le rôle, suffisant puisqu'il ne change qu'au login/logout).

TS `frontend/src/app/features/auth/auth.service.ts` (lignes 28-48)

```

/**
 * Authentifie l'utilisateur (format x-www-form-urlencoded attendu par
 * FastAPI/OAuth2),
 * stocke les tokens puis récupère le profil pour connaître son rôle réel.
 */
login(email: string, password: string): Observable<LoginResponse> {
    const body = new HttpParams().set('username', email).set('password', password);
    return this.http.post<LoginResponse>(`${this.apiUrl}/auth/login`, body, {
        headers: { 'Content-Type': 'application/x-www-form-urlencoded' }
    }).pipe(
        tap((response) => {
            localStorage.setItem('access_token', response.access_token);
            localStorage.setItem('refresh_token', response.refresh_token);
            this.isAuthenticated.set(true);
        }),
        // Le rôle vient toujours du profil renvoyé par le serveur, jamais d'une
        // supposition côté client.
        switchMap((response) => this.userService.me().pipe(
            tap((user) => localStorage.setItem('user_role', user.role)),
            map(() => response)
        ))
    );
}

```

`auth.interceptor.ts` — Intercepteur HTTP fonctionnel (`HttpInterceptorFn`) : ajoute l'en-tête `Authorization: Bearer <token>` à chaque requête sortante si un jeton existe ; en cas d'erreur 401 avec un jeton présent, déconnecte automatiquement l'utilisateur ; pour les requêtes GET en échec (hors 401), affiche un toast d'erreur générique (message différencié si `status === 0`, c.-à-d. serveur injoignable). Les requêtes d'écriture (POST/PATCH/DELETE) gèrent leur propre message d'erreur au niveau du composant — séparation assumée entre

erreurs de lecture (génériques) et erreurs d'écriture (contextuelles).

TS frontend/src/app/features/auth/auth.interceptor.ts (lignes 1-fin)

```
/**
 * Intercepteur HTTP global : ajoute le token Bearer à chaque requête sortante,
 * déconnecte automatiquement l'utilisateur sur une réponse 401, et affiche un
 * toast d'erreur générique pour les échecs de chargement (GET) silencieux.
 */
import { HttpResponse, HttpErrorResponse, HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { AuthService } from './auth.service';
import { ToastService } from '@shared/services/toast.service';

export const authInterceptor: HttpInterceptorFn = (request, next) => {
  const token = localStorage.getItem('access_token');
  const auth = inject(AuthService);
  const toast = inject(ToastService);
  return next(token ? request.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : request).pipe(
    catchError((error: unknown) => {
      if (error instanceof HttpErrorResponse) {
        if (token && error.status === 401) {
          auth.logout();
        } else if (request.method === 'GET' && error.status !== 401) {
          // Les échecs de lecture (GET) n'ont sinon aucun retour visuel nulle
          // part dans l'app.
          // Les actions (POST/PATCH/DELETE) gèrent déjà leur propre message au
          // niveau du composant.
          toast.error(
            error.status === 0
              ? 'Le serveur est indisponible. Réessayez dans un instant.'
              : 'Une erreur est survenue lors du chargement des données.'
          );
        }
      }
      return throwError(() => error);
    })
  );
};
```

login.component.ts — Écran unique de connexion/inscription (basculé via registerMode), formulaire *template-driven* ([ngModel]), pas de ReactiveFormsModule). Validation du mot de passe par expression régulière (12 caractères mini, minuscule + majuscule + chiffre) reflétée à la fois dans l'attribut HTML pattern et dans le message d'erreur. Le rôle du compte créé est toujours forcé à 'sportif' côté client — aucune UI ne permet de créer un compte coach depuis l'inscription. En cas de succès d'inscription, enchaîne automatiquement une connexion puis redirige vers /dashboard.

TS frontend/src/app/features/auth/login.component.ts (lignes 85-116)

```

/** Soumet le formulaire : inscription puis connexion automatique, ou connexion
    directe. */
submit(): void {
  this.error = '';
  this.success = '';
  const email = this.email.trim().toLowerCase();
  if (this.registerMode) {
    this.auth.register({ email, full_name: this.fullName.trim(), password:
      this.password, role: 'sportif' }).pipe(markForCheck(this.cd)).subscribe({
      next: () => {
        this.success = 'Compte créé. Connexion en cours...';
        this.auth.login(email,
          this.password).pipe(markForCheck(this.cd)).subscribe({
          next: () => void this.router.navigate(['/dashboard']),
          error: (response: { status: number }) => {
            this.error = response.status === 0
              ? 'Compte créé, mais le serveur est indisponible.'
              : 'Compte créé, mais la connexion automatique a échoué.';
          }
        });
      },
      error: (response: { status: number }) => {
        this.error = response.status === 409 ? 'Cette adresse email est déjà
          utilisée.' : 'Impossible de créer le compte. Vérifiez les
          informations saisies.';
      }
    });
    return;
  }
  this.auth.login(email, this.password).pipe(markForCheck(this.cd)).subscribe({
    next: () => void this.router.navigate(['/dashboard']),
    error: (response: { status: number }) => {
      this.error = response.status === 0
        ? 'Le serveur est indisponible. Démarrez l'API FastAPI sur le port 8000.'
        : 'Email ou mot de passe incorrect.';
    }
  });
}

```

7.2.3 Composants et services transverses (shared/)

shared/components/toast-container.component.ts — Composant d'affichage pur (app-toasts, monté une fois dans AppComponent), dérivé du signal ToastService.toasts(). Accessibilité : role="status", aria-live="polite" sur le conteneur.

TS frontend/src/app/shared/components/toast-container.component.ts (lignes 1-fin)

```

/**
 * Empilement visuel des notifications (toasts) émises par ToastService,
 * affiché en overlay au-dessus de toutes les pages.
 */
import { Component, inject } from '@angular/core';
import { ToastService } from '@shared/services/toast.service';

@Component({
  selector: 'app-toasts',
  standalone: true,
  template: `
    <div class="toast-stack" role="status" aria-live="polite">
      @for (toast of svc.toasts(); track toast.id) {
        <div class="toast" [class]="'k-' + toast.kind">
          <span class="toast-icon" aria-hidden="true">...</span>
          <span class="toast-msg">{{ toast.message }}</span>
          <button type="button" class="toast-close"
            (click)="svc.dismiss(toast.id)" aria-label="Fermer la
              notification">...</button>
        </div>
      }
    </div>
  `
})
/** Se contente de lire les toasts courants dans ToastService et de les rendre. */
export class ToastContainerComponent {
  readonly svc = inject(ToastService);
}

```

shared/services/toast.service.ts — État toasts = signal<Toast[]>([]) manipulé de façon immuable (update), avec disparition automatique après 4 secondes (setTimeout + dismiss). showOnNextLoad() sérialise un message dans sessionStorage pour l'afficher juste après un rechargement complet de page (location.reload()), cas où l'état en mémoire serait perdu — mécanisme pragmatique introduisant une seconde source de vérité (mémoire + sessionStorage) à garder à l'esprit.

TS frontend/src/app/shared/services/toast.service.ts (lignes 1-fin)

```

/**
 * Service Angular gérant les notifications toast affichées à l'utilisateur :
 * création, disparition automatique, et report d'un message via sessionStorage
 * pour l'afficher juste après un rechargement de page.
 */
import { Injectable, signal } from '@angular/core';

export type ToastKind = 'success' | 'error' | 'info';
export interface Toast { id: number; message: string; kind: ToastKind; }

const STORAGE_KEY = 'pending_toast';

/** File d'attente réactive des toasts actuellement affichés. */
@Injectable({ providedIn: 'root' })
export class ToastService {
  private nextId = 1;
  readonly toasts = signal<Toast[]>([]);
}

```

```

constructor() {
  // Si un toast a été programmé avant un location.reload() (via showOnNextLoad),
  // on le récupère et l'affiche une seule fois au démarrage du service.
  const pending = sessionStorage.getItem(STORAGE_KEY);
  if (pending) {
    sessionStorage.removeItem(STORAGE_KEY);
    const { message, kind } = JSON.parse(pending) as { message: string; kind:
      ToastKind };
    this.show(message, kind);
  }
}

success(message: string): void { this.show(message, 'success'); }
error(message: string): void { this.show(message, 'error'); }
info(message: string): void { this.show(message, 'info'); }

/** Affiche le message après le prochain chargement de page (à utiliser juste
    avant un location.reload()). */
showOnNextLoad(message: string, kind: ToastKind = 'success'): void {
  sessionStorage.setItem(STORAGE_KEY, JSON.stringify({ message, kind }));
}

/** Ajoute un toast à la pile et le retire automatiquement après 4 secondes. */
show(message: string, kind: ToastKind = 'info'): void {
  const id = this.nextId++;
  this.toasts.update((list) => [...list, { id, message, kind }]);
  setTimeout(() => this.dismiss(id), 4000);
}

dismiss(id: number): void {
  this.toasts.update((list) => list.filter((toast) => toast.id !== id));
}
}

```

shared/services/theme.service.ts — État theme = signal<'light' | 'dark'>, initialisé depuis localStorage sinon depuis la préférence système (window.matchMedia('(prefers-color-scheme: dark)').apply() pose l'attribut data-theme sur <html>, exploité par les variables CSS définies globalement (§2.9). Injection de DOCUMENT (jeton Angular) plutôt que l'objet global document, pattern recommandé pour la testabilité.

TS frontend/src/app/shared/services/theme.service.ts (lignes 1-fin)

```

/**
 * Service Angular gérant le thème clair/sombre de l'application : persistance
 * dans le localStorage, détection de la préférence système et application
 * au document via l'attribut `data-theme`.
 */
import { DOCUMENT } from '@angular/common';
import { Injectable, signal, inject } from '@angular/core';

export type Theme = 'light' | 'dark';

/** Centralise l'état du thème courant et sa persistance entre les sessions. */
@Injectable({ providedIn: 'root' })
export class ThemeService {
  private readonly document = inject(DOCUMENT);
  private readonly storageKey = 'sportplan-theme';
  readonly theme = signal<Theme>(this.initialTheme());

  constructor() {
    this.apply(this.theme());
  }

  /** Bascule entre thème clair et sombre, et mémorise le choix pour les
   prochaines visites. */
  toggle(): void {
    const next: Theme = this.theme() === 'light' ? 'dark' : 'light';
    this.theme.set(next);
    localStorage.setItem(this.storageKey, next);
    this.apply(next);
  }

  /** Thème au démarrage : préférence sauvegardée, sinon préférence système du
   navigateur. */
  private initialTheme(): Theme {
    const saved = localStorage.getItem(this.storageKey);
    if (saved === 'light' || saved === 'dark') {
      return saved;
    }
    return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' :
      'light';
  }

  /** Répercute le thème sur le DOM (attribut data-theme + color-scheme natif du
   navigateur). */
  private apply(theme: Theme): void {
    this.document.documentElement.dataset['theme'] = theme;
    this.document.documentElement.style.colorScheme = theme;
  }
}

```

shared/services/statistics.service.ts — Service minimal, une seule méthode mine(): Observable<Statistics> vers GET /statistics/me.

TS frontend/src/app/shared/services/statistics.service.ts (lignes 1-fin)

```

/**
 * Service Angular exposant les statistiques agrégées de l'utilisateur connecté
 * (séances, participations, performances) depuis l'API.
 */
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { API_URL } from '@core/api.config';

export interface Statistics {
  total_sessions: number;
  upcoming_sessions: number;
  total_participations: number;
  attended_sessions: number;
  total_performances: number;
  average_score: number | null;
}

/** Récupère les statistiques agrégées de l'utilisateur courant. */
@Injectable({ providedIn: 'root' })
export class StatisticsService {
  private readonly api = `${API_URL}/statistics`;
  constructor(private readonly http: HttpClient) {}

  mine(): Observable<Statistics> { return
    this.http.get<Statistics>(`${this.api}/me`); }
}

```

7.2.4 Tableau de bord

features/dashboard/dashboard.component.ts — Page d'accueil (commune à tous les rôles), qui agrège un aperçu de chaque module fonctionnel (séances, calendrier, participations, performances, objectifs, programmes, journal, notifications, et pour un coach/admin, sportifs suivis + création de séance) dans un même écran « module-rail ». Combine jusqu'à neuf flux Observable (combineLatest) pour construire ready\$ (bascule d'un squelette de chargement vers le contenu réel), avec shareReplay systématique pour éviter des appels HTTP dupliqués entre le template et les flux dérivés (ex. unreadCount\$ dérivé de notifications\$). computeTagline() construit un message d'accroche contextuel (priorité à un objectif dont l'échéance est à moins de 7 jours, sinon nombre de séances à venir). *Notable* : composant entièrement en lecture seule (aucune écriture), bon exemple de composition RxJS riche (combineLatest multiples, jointures manuelles participation<->séance et journal<->séance via Map+find) ; le contrôle d'accès aux modules coach reste côté client (auth.isCoachOrAdmin()), avec un commentaire assumant explicitement que le backend est la véritable barrière (403 sur /users/athletes) si ce contrôle client venait à être contourné.

TS frontend/src/app/features/dashboard/dashboard.component.ts (lignes 290-320)

```

private readonly allSessions$ = inject(SessionService).list().pipe(
  shareReplay({ bufferSize: 1, refCount: true })
);
private readonly sessions$ = this.allSessions$.pipe(
  map((sessions) => sessions
    .filter((session) => new Date(session.starts_at) > new Date())
    .sort((a, b) => new Date(a.starts_at).getTime() - new
      Date(b.starts_at).getTime())),
  shareReplay({ bufferSize: 1, refCount: true })
);
readonly nextSession$ = this.sessions$.pipe(map((sessions) => sessions[0]));
readonly upcomingSessions$ = this.sessions$.pipe(map((sessions) =>
  sessions.slice(0, 3)));

readonly participations$ = combineLatest([inject(ParticipationService).list(),
  this.allSessions$]).pipe(
  map([participations, sessions] => participations
    .slice(-3)
    .reverse()
    .map((participation) => ({
      ...participation,
      sessionTitle: sessions.find((session) => session.id ===
        participation.session_id)?.title ?? 'Séance',
    }))),
  shareReplay({ bufferSize: 1, refCount: true })
);

```

(illustre le style Observable du composant : un flux dérive le suivant par map/combineLatest, chacun mémorisé avec shareReplay pour n'être calculé/appelé qu'une fois même s'il est consommé à plusieurs endroits du template.)

7.2.5 Planning : séances, création d'entraînement, calendrier

features/sessions/session.service.ts — Façade HTTP des séances et de leurs exercices (list, create, update, delete, exercises, addExercise). *Notable* : un commentaire du code justifie l'usage systématique du slash final dans les URLs (`${apiUrl}/`) — sans lui, le serveur répondrait par une redirection 307 que les navigateurs ne suivent pas toujours de façon fiable pour une requête cross-origin porteuse d'un en-tête Authorization.

TS frontend/src/app/features/sessions/session.service.ts (lignes 1-fin)

```

/**
 * Service Angular pour les séances d'entraînement et leurs exercices :
 * consultation, création (coach) et gestion des exercices associés.
 */
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { API_URL } from '@core/api.config';

export interface SportSession {
  id: number; title: string; starts_at: string; coach_id: number; coach_name:
  string;
  duration_minutes: number; description?: string; capacity: number;
  registered_count: number;
}

export interface Exercise {
  id: number; session_id: number; name: string; description?: string;
  sets: number; repetitions?: number; rest_seconds: number;
}

/** CRUD des séances et de leurs exercices. */
@Injectable({ providedIn: 'root' })
export class SessionService {
  private readonly apiUrl = `${API_URL}/sessions`;
  constructor(private readonly http: HttpClient) {}

  // Avec le slash final : la route backend est déclarée "/", sans lui le serveur
  // répond
  // par une redirection 307 que les navigateurs ne suivent pas toujours de façon
  // fiable
  // pour une requête cross-origin avec en-tête Authorization.
  list(): Observable<SportSession[]> { return
    this.http.get<SportSession[]>(`${this.apiUrl}/`); }
  create(data: Omit<SportSession, 'id' | 'coach_name' | 'registered_count'>):
    Observable<SportSession> { return
    this.http.post<SportSession>(`${this.apiUrl}/`, data); }
  update(id: number, data: Partial<Pick<SportSession, 'title' | 'description' |
    'starts_at' | 'duration_minutes' | 'capacity'>>): Observable<SportSession> {
    return this.http.patch<SportSession>(`${this.apiUrl}/${id}`, data);
  }
  exercises(sessionId: number): Observable<Exercise[]> { return
    this.http.get<Exercise[]>(`${this.apiUrl}/${sessionId}/exercises/`); }
  addExercise(sessionId: number, data: Omit<Exercise, 'id' | 'session_id'>):
    Observable<Exercise> {
    return this.http.post<Exercise>(`${this.apiUrl}/${sessionId}/exercises/`,
    data);
  }
  delete(id: number): Observable<void> { return
    this.http.delete<void>(`${this.apiUrl}/${id}`); }
}

```

features/sessions/sessions.component.ts — Écran le plus riche du projet : liste des séances au style « module » (rail + panneau de détail, un pattern réutilisé ensuite pour Participations et Statistiques), éditeur intégré de création de séance (coach/admin, avec tableau d'exercices en FormArray), inscription, consultation des exercices avec chronomètre de repos, modification/annulation d'une séance, et panneau de suivi de présence (marquer

chaque inscrit Inscrit/Présent/Absent). Le rôle réel de l'utilisateur (nécessaire pour afficher l'éditeur et les actions de gestion) est obtenu via `UserService.me()`, jamais supposé depuis un jeton décodé côté client. Le chronomètre de repos (`timerSeconds`, `setInterval` exécuté hors zone Angular via `NgZone.runOutsideAngular` puis remis en zone pour la mise à jour d'affichage) illustre une gestion fine de la détection de changement pour un minuteur haute fréquence.

TS `frontend/src/app/features/sessions/sessions.component.ts` (lignes 481-493)

```
/** Démarre ou met en pause le décompte du chronomètre de repos. */
toggleTimer(): void {
  this.timerRunning = !this.timerRunning;
  if (this.timerRunning) this.timer = this.zone.runOutsideAngular(() =>
    setInterval(() => this.zone.run(() => {
      if (this.timerSeconds > 0) this.timerSeconds--; else { this.timerRunning =
        false; this.clearTimer(); }
      this.cd.markForCheck();
    }), 1000));
  else this.clearTimer();
}
resetTimer(): void { if (this.timerExercise) this.timerSeconds =
  this.timerExercise.rest_seconds; this.timerRunning = false;
  this.clearTimer(); }
get formattedTimer(): string { return `${Math.floor(this.timerSeconds /
  60).toString().padStart(2, '0')}:${(this.timerSeconds %
  60).toString().padStart(2, '0')}`; }
private clearTimer(): void { if (this.timer) clearInterval(this.timer);
  this.timer = undefined; }
```

(runOutsideAngular évite qu'Angular revérifie tout l'arbre de composants à chaque tic de setInterval; zone.run(...) ne rentre dans la zone que pour la fraction de code qui doit déclencher un rendu.)

`features/workout-create/workout-create.component.ts` — Écran dédié (coach) de composition d'une séance à partir d'une bibliothèque de plus de 60 exercices prédéfinis, organisée par **groupe musculaire** (`MuscleGroup` : jambes, fessiers & ischios, dos, pectoraux, épaules, bras, abdominaux, cardio & mobilité — huit onglets, chacun avec un pictogramme SVG distinct dessiné à la main : silhouette de jambes, bassin arrondi, torse en V, deux cercles pour les pectoraux, épaules arrondies, bras fléchi, grille « abdos », cœur avec pulsation). Le mouvement (`selectMuscle`) filtre la grille (`muscleExercises`, recalculée aussi par le champ de recherche libre). La création envoie la séance puis chaque exercice individuellement, avec un message dédié si la séance est créée mais qu'un exercice échoue.

TS `frontend/src/app/features/workout-create/workout-create.component.ts` (lignes 26-33)

```

type MuscleGroup = 'legs' | 'glutesHams' | 'back' | 'chest' | 'shoulders' | 'arms'
  | 'core' | 'cardio';
interface LibraryExercise { name: string; category: string; muscle: MuscleGroup; }

const MUSCLE_LABELS: Record<MuscleGroup, string> = {
  legs: 'Jambes', glutesHams: 'Fessiers & Ischios', back: 'Dos', chest:
    'Pectoraux',
  shoulders: 'Épaules', arms: 'Bras', core: 'Abdominaux', cardio: 'Cardio &
    Mobilité',
};
const MUSCLE_ORDER: MuscleGroup[] = ['legs', 'glutesHams', 'back', 'chest',
  'shoulders', 'arms', 'core', 'cardio'];

```

TS frontend/src/app/features/workout-create/workout-create.component.ts (lignes 248-255)

```

/** Exercices de la bibliothèque pour le groupe musculaire sélectionné, filtrés
    par le terme de recherche. */
get muscleExercises(): LibraryExercise[] {
  const term = this.searchTerm.trim().toLocaleLowerCase();
  return this.library.filter((exercise) =>
    exercise.muscle === this.selectedMuscle &&
    (!term || `${exercise.name}
      ${exercise.category}`.toLocaleLowerCase().includes(term))
  );
}

```

features/calendar/calendar.component.ts — Vue calendrier mensuelle (grille de 42 jours, lundi en première colonne), reconstruite localement (rebuild()) à chaque navigation de mois, avec regroupement des séances par jour dans une Map (accès O(1)). Permet de sélectionner un jour et de s'inscrire directement à une séance affichée. N'utilise pas AsyncPipe : état poussé dans des propriétés de classe avec markForCheck sur chaque souscription (à la différence du Dashboard, qui privilégie les flux Observable — les deux styles coexistent dans la base de code selon l'écran).

TS frontend/src/app/features/calendar/calendar.component.ts (lignes 126-148)

```

constructor() {
  this.loadSessions();
  // Connaître l'utilisateur courant et ses inscriptions pour afficher le badge
    "Inscrit".
  this.users.me().pipe(markForCheck(this.cd)).subscribe({
    next: (user) => {
      this.currentUserId = user.id;
      this.participationService.list().pipe(markForCheck(this.cd)).subscribe({
        next: (participations) => { this.participations = participations;
          this.rebuild(); },
        error: () => this.toast.error('Impossible de charger vos inscriptions.')
      });
    },
    error: () => this.toast.error('Impossible de charger votre profil.')
  });
  this.rebuild();
}

```

```

loadSessions(): void {
  this.sessionsLoadError = false;
  this.sessionService.list().pipe(markForCheck(this.cd)).subscribe({
    next: (sessions) => { this.sessions = sessions; this.rebuild(); },
    error: () => { this.sessionsLoadError = true; }
  });
}

```

(souscriptions imbriquées plutôt qu'un switchMap/combineLatest : fonctionnellement correct, mais moins idiomatique RxJS — cf. §7.2 synthèse.)

7.2.6 Suivi : participations, statistiques, objectifs, journal

features/participations/participation.service.ts — CRUD minimal (list, create, update du statut, remove).

TS frontend/src/app/features/participations/participation.service.ts (lignes 1-fin)

```

/** Service Angular pour les inscriptions (participations) d'un sportif à des
    séances. */
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { API_URL } from '@core/api.config';
export interface Participation { id: number; user_id: number; session_id: number;
    status: string; }

/** CRUD des participations (inscription, changement de statut, désinscription). */
@Injectable({ providedIn: 'root' })
export class ParticipationService {
  private readonly api = `${API_URL}/participations`;
  constructor(private readonly http: HttpClient) {}
  list(): Observable<Participation[]> { return
    this.http.get<Participation[]>(`${this.api}/`); }
  create(user_id: number, session_id: number): Observable<Participation> { return
    this.http.post<Participation>(`${this.api}/`, { user_id, session_id }); }
  update(id: number, status: string): Observable<Participation> { return
    this.http.patch<Participation>(`${this.api}/${id}`, { status }); }
  remove(id: number): Observable<void> { return
    this.http.delete<void>(`${this.api}/${id}`); }
}

```

features/participations/participations.component.ts — Écran « Participations », au même style modules que Séances. Le service GET /participations/ renvoie une portée différente selon le rôle (§6.2) ; le composant en tient compte : pour un coach/admin, chaque ligne affiche en plus le nom du sportif concerné (résolu via UserService.list()), uniquement appelé pour ces deux rôles puisque l'endpoint est réservé côté backend). Trois cartes de synthèse (à venir, terminées, taux de présence) et un état de chargement/erreur explicite complètent l'écran.

TS frontend/src/app/features/participations/participations.component.ts (lignes 170-208)

```

private load(): void {
  this.refresh$.pipe(
    switchMap(() => combineLatest([
      this.service.list(),
      this.sessions.list(),
      this.isManager ? this.users.list() : of([])
    ])),
    map(([participations, sessions, athletes]) => {
      const now = new Date();
      const athleteNames = new Map(athletes.map((athlete) => [athlete.id,
        athlete.full_name]));
      const enriched = participations.map((participation): EnrichedParticipation
        => {
        const session = sessions.find((s) => s.id === participation.session_id);
        const startsAt = session?.starts_at ?? '';
        return {
          ...participation,
          sessionTitle: session?.title ?? 'Séance supprimée',
          startsAt,
          durationMinutes: session?.duration_minutes ?? 0,
          coachName: session?.coach_name ?? '',
          athleteName: this.isManager ? (athleteNames.get(participation.user_id)
            ?? `Utilisateur #${participation.user_id}`) : null,
          upcoming: !!startsAt && new Date(startsAt) >= now,
        };
      });
      const upcoming = enriched.filter((item) => item.upcoming).sort((a, b) =>
        a.startsAt.localeCompare(b.startsAt));
      const past = enriched.filter((item) => !item.upcoming).sort((a, b) =>
        b.startsAt.localeCompare(a.startsAt));
      return { upcoming, past };
    }),
    markForCheck(this.cd)
  ).subscribe({
    next: ({ upcoming, past }) => {
      this.upcomingItems = upcoming;
      this.pastItems = past;
      this.items = [...upcoming, ...past];
      this.loading = false;
      this.loadError = false;
    },
    error: () => { this.loading = false; this.loadError = true; }
  });
}

```

(this.isManager ? this.users.list() : of([])) : l'appel réservé coach/admin n'est déclenché que pour ces rôles, jamais pour un sportif — cf. le 403 backend de GET /users/.)

features/performances/performance.service.ts — CRUD minimal (list, create).

TS frontend/src/app/features/performances/performance.service.ts (lignes 1-fin)

```

/** Service Angular pour les performances (scores) enregistrées lors des séances.
 */
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { API_URL } from '@core/api.config';
export interface Performance { id: number; user_id: number; session_id: number;
    score: number; notes?: string; recorded_at: string; }

/** Lecture et création des performances visibles par l'utilisateur connecté. */
@Injectable({ providedIn: 'root' })
export class PerformanceService {
    private readonly api = `${API_URL}/performances`;
    constructor(private readonly http: HttpClient) {}

    list(): Observable<Performance[]> { return
        this.http.get<Performance[]>(`${this.api}/`); }
    create(data: Omit<Performance, 'id' | 'recorded_at'>): Observable<Performance> {
        return this.http.post<Performance>(`${this.api}/`, data); }
}

```

features/performances/performances.component.ts — Écran « Statistiques », restructuré en trois modules accessibles à tous les rôles via le même rail que Séances/Participations : *Vue d'ensemble* (score moyen, meilleur score, graphique Chart.js de progression), *Historique* (liste détaillée des performances), *Assiduité* (taux de présence par séance passée — calculé sur les séances encadrées pour un coach/admin, sur les propres participations pour un sportif). Les graphiques (Chart, ligne pour la progression, barres pour la présence) lisent leurs couleurs directement depuis les variables CSS du thème courant (`getComputedStyle(document.documentElement)`), pour rester cohérents entre thème clair et sombre sans dupliquer de palette.

TS frontend/src/app/features/performances/performances.component.ts (lignes 212-229)

```

private loadAttendance(): void {
    combineLatest([this.sessionService.list(),
        this.participationService.list()]).pipe(markForCheck(this.cd)).subscribe(([
        sessions, participations]) => {
        const now = new Date();
        const ownSessionIds = new Set(participations.map((participation) =>
            participation.session_id));
        const managed = sessions.filter((session) => {
            if (new Date(session.starts_at) >= now) return false;
            return this.isCoach ? (this.isAdmin || session.coach_id ===
                this.currentUserId) : ownSessionIds.has(session.id);
        });
        this.attendancePoints = managed
            .map((session) => {
                const rows = participations.filter((participation) =>
                    participation.session_id === session.id);
                const present = rows.filter((participation) => participation.status ===
                    'present').length;
            });
    });
}

```

```

        return { title: session.title, date: session.starts_at, present, total:
            rows.length, rate: rows.length ? Math.round((present / rows.length)
                * 100) : 0 };
    })
    .sort((a, b) => new Date(a.date).getTime() - new Date(b.date).getTime());
    if (this.selectedSection === 'attendance' && this.attendancePoints.length)
        this.afterRender(() => this.renderAttendanceChart());
    });
}

```

(pour un sportif, rows ne contient que sa propre participation : rate vaut donc 100 ou 0 par séance, ce qui donne un historique présent/absent ; pour un coach/admin, rate moyenne la présence de tous les inscrits.)

features/goals/goal.service.ts et goals.component.ts — Gestion des objectifs personnels et des records personnels (deux ressources dans un même écran/service). Formulaires réactifs typés nonNullable, mise à jour optimiste locale des tableaux après chaque mutation. Barre de progression calculée côté client (current_value/target_value). *Notable* : chargement séquentiel (goals() puis, dans son callback, records()) plutôt qu'un forkJoin/combineLatest — fonctionnellement correct mais moins idiomatique.

TS frontend/src/app/features/goals/goal.service.ts (lignes 1-fin)

```

/** Service Angular pour les objectifs et records personnels du sportif connecté.
 */
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { API_URL } from '@core/api.config';

export interface Goal {
    id: number; user_id: number; title: string; metric: string; target_value: number;
    current_value: number; unit: string; due_date?: string; notes?: string;
}

export interface PersonalRecord {
    id: number; user_id: number; exercise_name: string; value: number; unit: string;
    achieved_at: string; notes?: string;
}

/** CRUD des objectifs et records personnels du sportif connecté. */
@Injectable({ providedIn: 'root' })

```

```

export class GoalService {
  private readonly api = API_URL;
  constructor(private readonly http: HttpClient) {}

  goals(): Observable<Goal[]> { return this.http.get<Goal[]>(`${this.api}/goals`); }
  createGoal(data: Omit<Goal, 'id' | 'user_id'>): Observable<Goal> { return
    this.http.post<Goal>(`${this.api}/goals`, data); }
  deleteGoal(id: number): Observable<void> { return
    this.http.delete<void>(`${this.api}/goals/${id}`); }
  records(): Observable<PersonalRecord[]> { return
    this.http.get<PersonalRecord[]>(`${this.api}/records`); }
  createRecord(data: Omit<PersonalRecord, 'id' | 'user_id' | 'achieved_at'>):
    Observable<PersonalRecord> { return
      this.http.post<PersonalRecord>(`${this.api}/records`, data); }
}

```

TS frontend/src/app/features/goals/goals.component.ts (lignes 72-85)

```

load(): void {
  this.loading = true;
  this.loadError = false;
  this.service.goals().pipe(markForCheck(this.cd)).subscribe({
    next: (goals) => {
      this.goals = goals;
      this.service.records().pipe(markForCheck(this.cd)).subscribe({
        next: (records) => { this.records = records; this.loading = false; },
        error: () => { this.loadError = true; this.loading = false; }
      });
    },
    error: () => { this.loadError = true; this.loading = false; }
  });
}

```

features/journal/journal.service.ts et journal.component.ts — Saisie d'un bilan post-séance (fatigue 1-10, humeur, douleur, notes), rattaché à une séance existante via un mat-select peuplé par SessionService. L'historique affiche aussi le commentaire du coach (coach_comment), confirmant le flux de collaboration sportif/coach autour d'une même entrée (cf. routers/journal.py, §7.1.4). *Notable* : composition de deux services indépendants (JournalService + SessionService) via deux souscriptions distinctes plutôt qu'un flux combiné.

TS frontend/src/app/features/journal/journal.service.ts (lignes 1-fin)

```

/**
 * Service Angular pour le journal d'entraînement (bilans post-séance :
 * fatigue, humeur, douleur, commentaire du coach).
 */
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { API_URL } from '@core/api.config';

export interface TrainingJournal {
  id: number; user_id: number; session_id: number; notes?: string; fatigue: number;
  mood: string; pain?: string; coach_comment?: string; created_at: string;
  updated_at: string;
}

/** Lecture et création des entrées de journal du sportif connecté. */
@Injectable({ providedIn: 'root' })
export class JournalService {
  private readonly api = `${API_URL}/journal`;
  constructor(private readonly http: HttpClient) {}

  list(): Observable<TrainingJournal[]> { return
    this.http.get<TrainingJournal[]>(this.api); }

  create(data: Omit<TrainingJournal, 'id' | 'user_id' | 'created_at' |
    'updated_at'>): Observable<TrainingJournal> {
    return this.http.post<TrainingJournal>(this.api, data);
  }
}

```

TS frontend/src/app/features/journal/journal.component.ts (lignes 86-98)

```

/** Envoie le bilan de séance à l'API et l'ajoute directement à l'historique,
    sans recharger la page. */
create(): void {
  if (this.form.invalid) return;
  this.error = '';
  this.service.create(this.form.getRawValue()).pipe(markForCheck(this.cd)).subscribe({
    next: (entry) => {
      this.journals = [entry, ...this.journals];
      this.form.reset({ session_id: 0, fatigue: 5, mood: 'bien', notes: '',
        pain: '' });
      this.toast.success('Bilan enregistré.');
```

7.2.7 Ressources : programmes et notifications

features/programs/program.service.ts et programs.component.ts — Création (nom, description, nombre de semaines 1-52) et liste de programmes d'entraînement, sans édition ni suppression de séances modèles depuis cet écran (le champ sessions de l'API,

prévu pour contenir des séances types, reste toujours vide à la création côté frontend — fonctionnalité de l'API non exploitée par cet écran).

TS frontend/src/app/features/programs/program.service.ts (lignes 1-fin)

```
/** Service Angular pour les programmes d'entraînement (modèles réutilisables sur
    plusieurs semaines). */
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { API_URL } from '@core/api.config';

export interface WorkoutProgram {
  id: number; user_id: number; name: string; description?: string; weeks: number;
  sessions: Array<{ title: string; focus?: string; exercises?: string[] }>;
  created_at: string;
}

/** CRUD des programmes d'entraînement de l'utilisateur connecté. */
@Injectable({ providedIn: 'root' })
export class ProgramService {
  private readonly api = `${API_URL}/programs`;
  constructor(private readonly http: HttpClient) {}
  list(): Observable<WorkoutProgram[]> { return
    this.http.get<WorkoutProgram[]>(this.api); }
  create(data: Omit<WorkoutProgram, 'id' | 'user_id' | 'created_at'>):
    Observable<WorkoutProgram> { return this.http.post<WorkoutProgram>(this.api,
      data); }
  delete(id: number): Observable<void> { return
    this.http.delete<void>(`${this.api}/${id}`); }
}
```

TS frontend/src/app/features/programs/programs.component.ts (lignes 64-76)

```
/** Crée un programme vide (sans séances modèles) et l'ajoute directement à la
    liste. */
create(): void {
  if (this.form.invalid) return;
  this.createError = '';
  this.service.create({ ...this.form.getRawValue(), sessions: []
    }).pipe(markForCheck(this.cd)).subscribe({
    next: (program) => {
      this.programs = [program, ...this.programs];
      this.form.reset({ name: '', description: '', weeks: 4 });
      this.toast.success('Programme créé.');
```

features/notifications/notification.service.ts et notifications.component.ts — Centre d'alertes en lecture seule + action « marquer comme lu » (PATCH /notifications/{id}/read). Icône dérivée du champ kind par un caractère unicode plutôt qu'un composant d'icône Material — léger écart de cohérence visuelle avec le reste de l'application.

TS frontend/src/app/features/notifications/notification.service.ts (lignes 1-fin)

```
/** Service Angular pour les notifications utilisateur : liste et marquage comme lu. */
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { API_URL } from '@core/api.config';

export interface AppNotification {
  id: number; user_id: number; title: string; message: string; kind: string;
  is_read: boolean; created_at: string;
}

/** Lecture et mise à jour de l'état "lu" des notifications de l'utilisateur connecté. */
@Injectable({ providedIn: 'root' })
export class NotificationService {
  private readonly api = `${API_URL}/notifications`;
  constructor(private readonly http: HttpClient) {}
  list(): Observable<AppNotification[]> { return
    this.http.get<AppNotification[]>(this.api); }
  markRead(id: number): Observable<AppNotification> { return
    this.http.patch<AppNotification>(`${this.api}/${id}/read`, {}); }
}
```

TS frontend/src/app/features/notifications/notifications.component.ts (lignes 50-58)

```
/** Marque la notification comme lue côté serveur et met à jour la liste en place, sans recharger la page. */
read(notification: AppNotification): void {
  this.service.markRead(notification.id).pipe(markForCheck(this.cd)).subscribe({
    next: (updated) => {
      notification.is_read = updated.is_read;
      this.toast.success('Notification marquée comme lue.');
```

7.2.8 Équipe et compte : sportifs suivis, administration, profil

features/athletes/user.service.ts — Façade HTTP pure du profil utilisateur (me, athletes, list, update), sans logique métier ; la restriction d'accès à list()/athletes() repose entièrement sur le backend (403 attendu) et sur les guards de route, pas sur le service.

TS frontend/src/app/features/athletes/user.service.ts (lignes 1-fin)

```

/**
 * Service Angular pour le profil utilisateur : récupération du compte
 * courant, liste des sportifs suivis par un coach, et mise à jour du profil.
 */
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { API_URL } from '@core/api.config';

export interface User {
  id: number; email: string; full_name: string; role: string; is_active: boolean;
}

/** Gère la lecture/écriture du profil utilisateur. */
@Injectable({ providedIn: 'root' })
export class UserService {
  private readonly api = `${API_URL}/users`;
  constructor(private readonly http: HttpClient) {}
  /** Profil de l'utilisateur connecté. */
  me(): Observable<User> { return this.http.get<User>(`${this.api}/me`); }

  /** Liste des sportifs visibles par un coach. */
  athletes(): Observable<User[]> { return
    this.http.get<User[]>(`${this.api}/athletes`); }

  /** Liste tous les comptes (réservé admin, pour la gestion des comptes). */
  list(): Observable<User[]> { return this.http.get<User[]>(`${this.api}/`); }

  /** Met à jour un sous-ensemble éditable du profil (nom, rôle, statut actif). */
  update(id: number, data: Partial<Pick<User, 'full_name' | 'role' |
    'is_active'>>): Observable<User> {
    return this.http.patch<User>(`${this.api}/${id}`, data);
  }
}

```

features/athletes/athletes.component.ts — Liste des sportifs suivis (coach), avec taux de présence et dernière activité calculés côté client en croisant trois flux (combineLatest sur athletes(), participations.list(), sessions.list()).

TS frontend/src/app/features/athletes/athletes.component.ts (lignes 77-100)

```

readonly athletes$ = combineLatest([
  this.users.athletes(),
  this.participationService.list(),
  this.sessionService.list()
]).pipe(
  map(([athletes, participations, sessions]): AthleteSummary[] => {
    const sessionsById = new Map(sessions.map((session) => [session.id,
      session]));
    return athletes.map((athlete) => {
      const own = participations.filter((participation) => participation.user_id
        === athlete.id);
      const attended = own.filter((participation) => participation.status ===
        'present').length;
      const lastSession = own
        .map((participation) => sessionsById.get(participation.session_id))
        .filter((session): session is NonNullable<typeof session> => !!session)
        .sort((a, b) => new Date(b.starts_at).getTime() - new
          Date(a.starts_at).getTime())[0];
      return {
        ...athlete,
        totalSessions: own.length,
        sessionsAttended: attended,
        attendanceRate: own.length ? Math.round((attended / own.length) * 100) :
          0,
        lastActivity: lastSession ? new
          Date(lastSession.starts_at).toLocaleDateString('fr-FR', { day:
            '2-digit', month: '2-digit', year: 'numeric' }) : null,
      };
    });
  })
);

```

features/athletes/athlete-detail.component.ts — Fiche individuelle d'un sportif (route /athletes/:id), recalculant une logique d'assiduité très proche de celle d'AthletesComponent — une factorisation dans un service dédié serait pertinente (duplication relevée). Récupère l'athlète via la liste complète des sportifs puis un .find() côté client plutôt qu'un endpoint dédié par identifiant (sur-récupération mineure).

TS frontend/src/app/features/athletes/athlete-detail.component.ts (lignes 77-109)

```

constructor() {
  const athleteId = Number(this.route.snapshot.paramMap.get('id'));
  forkJoin([
    this.users.athletes(),
    this.participationService.list(),
    this.performanceService.list(),
    this.sessionService.list(),
  ]).pipe(markForCheck(this.cd)).subscribe(([athletes, participations,
    performances, sessions]) => {
    this.athlete = athletes.find((item) => item.id === athleteId);
    this.loaded = true;
    if (!this.athlete) return;

    const sessionsById = new Map(sessions.map((session) => [session.id,
      session]));
    const ownParticipations = participations.filter((participation) =>
      participation.user_id === athleteId);
    this.totalSessions = ownParticipations.length;
    this.sessionsAttended = ownParticipations.filter((participation) =>
      participation.status === 'present').length;
    this.attendanceRate = this.totalSessions ? Math.round((this.sessionsAttended
      / this.totalSessions) * 100) : 0;

    const lastSession = ownParticipations
      .map((participation) => sessionsById.get(participation.session_id))
      .filter((session): session is NonNullable<typeof session> => !!session)
      .sort((a, b) => new Date(b.starts_at).getTime() - new
        Date(a.starts_at).getTime())[0];
    this.lastActivity = lastSession
      ? new Date(lastSession.starts_at).toLocaleDateString('fr-FR', { day:
        '2-digit', month: '2-digit', year: 'numeric' })
      : null;

    this.performanceScores = performances
      .filter((performance) => performance.user_id === athleteId)
      .sort((a, b) => new Date(a.recorded_at).getTime() - new
        Date(b.recorded_at).getTime())
      .slice(-5)
      .map((performance) => performance.score);
  });
}

```

(logique de calcul d'assiduité quasi identique à celle d'AthletesComponent ci-dessus — duplication candidate à factorisation, cf. §10.3.)

features/admin/admin-users.component.ts — Gestion des comptes (admin) : changement de rôle (mat-select) et activation/désactivation (mat-slide-toggle) par utilisateur, avec protection explicite empêchant l'admin de modifier son propre compte depuis cet écran (vérifiée côté template ; la garantie réelle reste côté backend, cf. §6.2).

TS frontend/src/app/features/admin/admin-users.component.ts (lignes 83-106)

```

private refresh$ = this.service.me();
readonly users$ = this.refresh$.pipe(
  switchMap((me) => {
    this.currentUserId = me.id;
    return this.service.list();
  }),
  shareReplay({ bufferSize: 1, refCount: true })
);

// ...

changeRole(user: User, role: string): void {
  if (role === user.role) return;
  this.service.update(user.id, { role }).pipe(markForCheck(this.cd)).subscribe({
    next: () => { user.role = role; this.toast.success(`${user.full_name} est maintenant ${this.roleLabel(role).toLowerCase()}.`); },
    error: () => this.toast.error('Impossible de changer le rôle de ce compte.')
  });
}

```

features/profile/profile.component.ts — Profil personnel commun à tous les rôles : modification du nom (l'email reste non éditable, cohérent avec le typage de `UserService.update`), consultation des statistiques agrégées (dégradées proprement en cas d'échec via `catchError(() => of(null))`), bascule du thème, et changement de mot de passe (sans champ de confirmation du nouveau mot de passe, à noter comme choix d'UX discutable).

TS frontend/src/app/features/profile/profile.component.ts (lignes 101-119)

```

/** Enregistre le nom complet modifié via UserService.update. */
save(id: number): void {
  if (this.form.invalid) return;
  this.saving = true; this.message = '';
  this.service.update(id,
    this.form.getRawValue()).pipe(markForCheck(this.cd)).subscribe({
    next: () => { this.saving = false; this.message = 'Profil mis à jour.'; },
    error: () => { this.saving = false; this.message = 'La mise à jour a échoué. Réessayez.'; }
  });
}

/** Change le mot de passe puis réinitialise le formulaire en cas de succès. */
changePassword(): void {
  if (this.passwordForm.invalid) return;
  const { current_password, new_password } = this.passwordForm.getRawValue();
  this.auth.changePassword(current_password,
    new_password).pipe(markForCheck(this.cd)).subscribe({
    next: () => { this.passwordForm.reset(); this.passwordMessage = 'Mot de passe modifié avec succès.'; },
    error: (error) => { this.passwordMessage = error?.error?.detail || 'Impossible de modifier le mot de passe.'; }
  });
}

```

Synthèse transversale du frontend : deux styles de gestion d'état coexistent selon l'écran — `Observable` + `AsyncPipe` + `combineLatest` (Dashboard, Profil, Athlètes) d'une part, état impératif en propriétés de classe + `markForCheck` (Calendrier, Fiche athlète) d'autre part ; le contrôle d'accès par rôle repose de façon cohérente sur trois couches (guards de route, conditions d'affichage dans les templates, et vérification finale côté backend) ; une même logique de calcul d'assiduité est dupliquée entre deux composants « Athlètes », un bon candidat de factorisation pour une évolution future.

7.2.9 Styles globaux et système de design (`styles.css`)

Le fichier `frontend/src/styles.css` centralise tout le design visuel de l'application (aucun composant n'a de feuille de style propre — cohérent avec les templates 100 % inline).

Points clés :

- **Thématisation** : palette définie une fois sous `:root` (thème clair) puis redéfinie sous `html[data-theme='dark']` — toute règle qui référence `var(--accent-primary)` etc. change de valeur instantanément selon l'attribut posé par `ThemeService` (§2.9, §7.2.3), sans dupliquer la moindre règle entre les deux thèmes.
- **Système de teinte modulaire (`--module-tint`)** : les pages « module » (Séances, Participations, Statistiques, tableau de bord) partagent un jeu de classes `.c-primary/.c-secondary/.c-success/.c-warning/.c-info/.c-danger` qui fixent une variable CSS locale `--module-tint`, consommée par les règles de fond, de bordure et d'icône du rail et du panneau de détail — une seule déclaration de couleur par élément, appliquée de façon cohérente à l'état par défaut, au survol et à l'état actif.
- **Pattern « module-shell »** : `module-shell/module-rail/module-detail/module-detail-card` forment un gabarit réutilisé par plusieurs écrans (rail de sélection à gauche, panneau de détail à droite), au lieu d'un composant Angular générique dédié — la réutilisation se fait ici entièrement au niveau CSS plutôt que par un composant partagé.
- **Accessibilité et thème sombre** : le contraste des puces d'icônes (`--bg-surface-alt` vs teintes `--accent-secondary/--module-tint`) a été retravaillé pendant le développement du projet (voir le choix documenté au §10.3) pour rester lisible sur fond clair comme sur fond sombre.

8.1 Backend (backend/tests/, exécutés avec pytest)

- `conftest.py` : fixe les variables d'environnement nécessaires (`DATABASE_URL` sur une base SQLite locale, `SECRET_KEY`) avant tout import de l'application, afin que les tests ne dépendent jamais d'une vraie instance SQL Server.
- `test_schemas.py` : vérifie que les contraintes déclarées dans les schémas Pydantic rejettent bien les valeurs invalides — un objectif à valeur cible nulle (`GoalCreate(target_value=0)`) lève une `ValidationError` ; un niveau de fatigue de journal dans la plage valide (1-10) est accepté, hors plage (`fatigue=11`) est rejeté ; un programme de plus de 52 semaines est rejeté. Ces tests documentent et figent le comportement de validation attendu (§2.5).
- `test_security.py` : vérifie que le mot de passe haché diffère du mot de passe en clair et que `verify_password` accepte le bon mot de passe et rejette un mauvais (§2.6.1) ; qu'un jeton d'accès contient bien `sub` et `type: "access"` ; et qu'un jeton expiré (durée de vie négative construite exprès) est rejeté par `decode_token` (`JWTError`).
- `test_time.py` : vérifie que `is_past` distingue correctement une date passée d'une date future.

`pytest.ini` limite l'exécution au dossier `tests/` et ajoute la racine du projet au `PYTHONPATH` (`pythonpath = .`), pour que `import app...` fonctionne sans installation du paquet.

8.2 Frontend (* .spec.ts, exécutés avec vitest)

- `theme.service.spec.ts` : vérifie le thème par défaut (`light` en l'absence de préférence stockée), et que `toggle()` bascule l'état, le persiste dans `localStorage`, et met à jour l'attribut `data-theme` du document.
- `toast.service.spec.ts` : vérifie la création d'un toast pour chaque niveau de sévérité, la suppression manuelle (`dismiss`), la disparition automatique après 4 secondes (minuteurs simulés via `vi.useFakeTimers()` — un test de comportement asynchrone sans attendre réellement le délai), et le mécanisme `sessionStorage` de report d'un toast après rechargement de page.
- `auth.service.spec.ts` : utilise `HttpTestingController` (double de test HTTP fourni par Angular) pour vérifier qu'une connexion réussie enchaîne bien un appel à `/users/me` et persiste les jetons et le rôle dans `localStorage`, et que `changePassword()` poste le bon corps de requête au bon endpoint — un test d'intégration léger qui ne contacte jamais le vrai backend (`http.verify()` s'assure qu'aucune requête imprévue n'a été émise).

- `dashboard.component.spec.ts` : instancie `DashboardComponent` avec tous ses services injectés remplacés par des doublures minimales (`{ provide: ..., useValue: { list: () => of([]) } }`), pattern classique d'**injection de dépendances au service des tests** (§2.8) — vérifie que les modules réservés coach/admin n'apparaissent que pour ce rôle, que `selectModule()` change bien la sélection, et plusieurs fonctions pures du composant (`firstName`, `avgScore`, `goalProgress`, `sparkBars`, `statusLabel`) avec leurs cas limites (moyenne vide, progression plafonnée à 100 %, valeurs de score aberrantes écrêtées).

Le choix des quatre fichiers testés côté frontend (thème, toasts, authentification, tableau de bord) couvre un état global partagé (thème/toasts), un flux métier sensible (authentification), et le composant le plus complexe en composition de données (tableau de bord) — cohérent avec la même logique de priorisation par risque que côté backend (§2.10).

9. Rôles et permissions — synthèse

09

Trois rôles sont stockés dans `users.role` : `sportif` (rôle par défaut à l'inscription), `coach`, `admin`. Le frontend ne fait jamais confiance à une valeur de rôle mise en cache côté client pour la sécurité : chaque écran protégé correspond à un endpoint qui revérifie le rôle côté serveur (§2.6.3, §6.2).



Action	Sportif	Coach	Admin
S'inscrire à une séance	Oui	Oui	Oui
Créer une séance	Non	Oui (les siennes)	Oui
Modifier/annuler une séance	Non	Oui (les siennes)	Oui (toutes)
Marquer une présence	Non	Oui (ses séances)	Oui (toutes)
Voir la liste des sportifs	Non	Oui	Oui
Voir l'assiduité de ses propres séances (Statistiques)	Oui	Oui (celle de ses sportifs)	Oui (toutes)
Voir le nom du sportif sur une participation	— (déjà le sien)	Oui	Oui
Gérer les comptes (rôle, actif/inactif)	Non	Non	Oui

Un sportif ne peut consulter et modifier que ses propres données (séances, participations, performances, objectifs, programmes, journal) ; un coach hérite de tout ce qu'un sportif peut faire, avec en plus la gestion de ses propres séances et le suivi de ses sportifs ; un admin hérite de tout ce qu'un coach peut faire, sans restriction de propriété, plus la gestion globale des comptes.

10. Choix de conception et limites connues

10

Un dossier technique honnête ne se limite pas à décrire ce qui fonctionne : cette section rassemble, en un seul endroit, les observations de qualité relevées pendant la revue de code (chapitre 7), classées par nature.

10.1 Défauts fonctionnels confirmés

- `GET /statistics/me` (`routers/statistics.py`) : le champ `average_score` renvoyé à un coach ou un admin est calculé sur l'ensemble des performances de **tous** les utilisateurs du système, alors que les autres métriques de la même réponse sont bien restreintes à son propre périmètre (ses séances, ou toutes pour un admin) — un score moyen non représentatif pourrait induire en erreur.
- `POST /notifications` (`routers/notifications.py`) : la notification créée par un admin cible toujours l'admin appelant plutôt qu'un destinataire fourni dans la requête — défaut reconnu par un commentaire dans le code source lui-même.

10.2 Limites techniques assumées (acceptables pour un projet scolaire)

- **Absence de migrations de schéma versionnées** : les tables sont créées par introspection des modèles (`Base.metadata.create_all`) au démarrage, sans outil comme Alembic — suffisant tant que le schéma n'évolue pas sur une base déjà peuplée en production.
- **Rate limiting en mémoire** (`rate_limit.py`) : non partagé entre plusieurs instances du backend, réinitialisé à chaque redémarrage — documenté comme tel dans le code.
- **Conditions de concurrence théoriques** : les vérifications « séance complète », « déjà inscrit » ou « entrée de journal déjà existante » suivent le schéma `SELECT` puis `INSERT` sans verrou explicite ; deux requêtes quasi simultanées pourraient, en théorie, franchir la même limite. Une mise en production à fort trafic simultané nécessiterait une contrainte d'unicité en base et/ou une transaction verrouillante.
- **URL de l'API codée en dur côté frontend** (`core/api.config.ts`) : pas de distinction d'environnement (développement/production) — à résoudre via les fichiers `environment.ts` standards d'Angular avant un déploiement réel.
- **localStorage pour les jetons JWT** : plus simple à implémenter que des cookies `HttpOnly`, mais accessible à n'importe quel script exécuté dans la page (surface d'attaque XSS à garder à l'esprit si une dépendance tierce compromise venait à injecter du code).

10.3 Incohérences mineures de convention

- Écarts de contrainte entre schéma Pydantic (strict) et colonne SQLAlchemy (permissive) sur `Exercise.sets`, ou l'inverse (chaîne libre côté schéma, `Enum SQL` côté modèle) sur `role` et `status` — cf. §5.3.
- Deux stratégies de suppression en cascade coexistent (ORM vs base de données) — cf. §5.2.
- Le filtrage par rôle (admin/coach/sportif) est réécrit à l'identique dans quatre routeurs différents plutôt que factorisé.
- Deux styles de gestion d'état réactif coexistent côté frontend (`Observable+AsyncPipe` vs état impératif + `markForCheck`), selon l'écran plutôt que selon une règle explicite.
- Une même logique de calcul d'assiduité est dupliquée entre `AthletesComponent` et `AthleteDetailComponent`.
- La conception visuelle a elle-même itéré en cours de projet : une première version des puces d'icônes de la barre latérale utilisait une teinte trop proche du fond de page (contraste insuffisant), corrigée en reprenant la palette teintée déjà utilisée pour les pictogrammes d'exercices ; une tentative de teinter chaque groupe musculaire d'une couleur différente a ensuite été délibérément annulée au profit d'une couleur unique, sur retour explicite — un exemple concret d'itération de design piloté par la préférence utilisateur plutôt que par une règle théorique absolue.

Aucune de ces observations ne remet en cause la cohérence d'ensemble du projet : elles indiquent des axes d'amélioration réalistes pour une prochaine itération, ce qui est la nature même d'une revue de code.

- Introduire des migrations de schéma versionnées (Alembic) pour préparer une évolution du modèle de données sans perte en production.
- Factoriser le filtrage par rôle (admin/coach/sportif), répété dans plusieurs routeurs, dans une fonction utilitaire commune côté backend.
- Ajouter une contrainte d'unicité en base sur (`user_id`, `session_id`) pour `participations`, à l'image de celle déjà présente sur `training_journals`, pour fermer la fenêtre de concurrence théorique.
- Distinguer les environnements de build côté frontend (`environment.ts/environment.prod.ts`) plutôt qu'une URL d'API codée en dur.
- Étendre la couverture de tests automatisés aux routeurs FastAPI eux-mêmes (tests d'intégration avec une base SQLite éphémère et le client de test de FastAPI), au-delà des schémas et de la sécurité déjà couverts.
- Partager la logique de calcul d'assiduité (actuellement dupliquée entre deux composants Angular) dans un service dédié.
- Explorer une application mobile ou un partage de séances avec un coach externe, pistes déjà identifiées dans le support de présentation du projet.

SportPlan illustre, sur un périmètre fonctionnel volontairement complet (séances, participations, performances, objectifs, programmes, journal, notifications, gestion multi-rôles), l'ensemble des concepts théoriques attendus d'une application web moderne à deux niveaux : une architecture client-serveur strictement séparée communiquant par une API REST, une persistance relationnelle via un ORM, une validation systématique des données aux frontières du système, une sécurité construite en profondeur (hachage Argon2, JWT avec rotation, RBAC, revalidation serveur systématique), et une interface réactive construite sur les patterns modernes d'Angular (composants standalone, signals, RxJS).

La revue de code fichier par fichier (chapitre 7) montre un code globalement cohérent et lisible, dont les quelques défauts identifiés (chapitre 10) sont mineurs, localisés, et pour la plupart déjà documentés dans le code lui-même — un signe, en soi, d'un développement conscient de ses propres limites plutôt que d'un manque de rigueur. C'est précisément ce type de lecture critique, théorique autant que pratique, qu'un dossier technique de fin de session doit être en mesure de produire.